

Campus Indoor AR Navigation

Final Project Report

Team: MCS03

Chong Jia Yee | 33563888 | jcho0156@student.monash.edu

Wong Jia Cheng | 32951159 | jwon0147@student.monash.edu

Beh Hanyu | 33607265 | hbeh0003@student.monash.edu

Soo Wooi King | Supervisor | soo.wooking@monash.edu

FIT3162: Computer Science & Data Science Project

October 28, 2024

Word Count: 12430 Words

Table of Contents

1. Introduction.....	5
1.1 Project Overview.....	5
1.2 Report Overview.....	6
2. Project Background.....	7
2.1 Background.....	7
2.1.1 The Evolution of Indoor Navigation Technologies Using Smartphones.....	7
2.1.2 Transition to Bluetooth-Based Real-Time Positioning.....	7
2.2 Rationale.....	8
2.2.1 Real-Time Positioning.....	8
2.2.2 Orientation.....	8
2.2.3 Shortest Route.....	8
2.3 Literature Review.....	9
2.3.1 Indoor Positioning Techniques.....	9
2.3.1.1 Wi-Fi Fingerprinting.....	9
2.3.1.2 Bluetooth Beacons.....	9
2.3.2 Shortest Path Implementation.....	10
2.3.2.1 Dijkstra's Algorithm.....	10
2.3.2.2 A Algorithm*.....	10
2.3.2.3 Breadth-First Search (BFS).....	10
2.3.3 User Orientation.....	11
2.3.3.1 Device Geomagnetic Sensors.....	11
2.3.3.2 Inertial Measurement Units (IMUs).....	11
2.3.3.3 Visual-Inertial Odometry (VIO).....	11
2.4 Conclusion.....	12
3. Outcomes.....	13
3.1 What Has Been Implemented.....	13
3.1.1 Automatic Indoor Position Detection (Localization).....	13
3.1.2 Shortest Path Calculation.....	13
3.1.3 Dynamic Rerouting.....	14
3.1.4 Student Account Integration.....	14
3.1.5 Multi-Floor Navigation.....	14
3.1.6 Voice Navigation with Multi-Lingual Support.....	14
3.1.7 ETA And Distance Visualisation.....	14
3.1.8 Orientation Sensing.....	15
3.2 Product Delivered.....	16
3.2.1 Automatic Indoor Position Detection (Localization).....	16
3.2.2 Shortest Path Calculation / Eta and Distance Visualisation / Voice Navigation with Multi-Lingual Support.....	17
3.2.3 Student Account Integration.....	18
3.2.4 Orientation Sensing.....	19
3.2.5 Dynamic Rerouting.....	20
3.2.5 Multi-Floor Navigation.....	21

3.3 How Project Requirements Are Met.....	22
3.4 Justification of Decisions Made.....	27
3.5 Limitations of Project Outcomes.....	30
3.5.1 Small coverage of campus map.....	30
3.5.2 Long response time in updating user's position.....	30
3.5.3 Longer Loading of Route During High Volume of Traffic.....	31
3.5.4 No AR Implementation For Camera View.....	31
3.6 Potential Improvements and Future Work.....	32
3.6.1 Scaling up the map.....	32
3.6.2 Rent server that can afford higher traffic load.....	32
3.6.3 Implementation of AR feature for camera view.....	32
3.6.4 Expanding Beacon Coverage and Enhancing Localization Accuracy.....	33
3.6.5 Improving Response Time Through Beacon Frequency Adjustments.....	33
3.6.6 User Feedback Mechanisms for Continuous Improvement.....	33
3.6.7 Allow users to have more customization.....	34
3.6.8 Event Notifications and Navigation Integration.....	34
4. Methodology.....	35
4.1 System Architecture.....	35
4.1.1 Overview.....	35
4.1.2 Deviation.....	36
4.2 Map Planning.....	37
4.2.1 Overview.....	37
4.2.2 Deviation.....	37
4.3 User Positioning.....	38
4.3.1 Training/Offline Phase.....	38
4.3.2 Positioning/Online Phase.....	38
4.4 Route Planning.....	40
4.4.1 Data Structure Parsing.....	40
4.4.2 Shortest Path Algorithm.....	40
4.4.3 Deviation.....	41
4.5 Path Guiding.....	42
4.5.1 2-Dimensional Based.....	42
4.5.2 Audio Based.....	42
4.5.3 Deviation.....	43
4.6 Software & Hardware Specification.....	44
5. Software Deliverables.....	45
5.1 Summary of software deliverables.....	45
5.2 Software Quality Summary.....	49
5.2.1 Robustness.....	49
5.2.2 Security.....	50
5.2.3 Usability.....	52
5.2.4 Scalability.....	54
5.2.5 Portability.....	56
5.2.6 Maintainability.....	58

5.3 Sample Source Code.....	59
6. Software And Project Critique.....	60
6.1 Discussion of Project Execution.....	60
6.1.1 Success.....	60
6.1.2 Failures.....	60
6.2 Comparison of Realised Project to Initial Project Proposal.....	62
7. Conclusion.....	63
8. References.....	64
9. Appendix.....	67

1. Introduction

1.1 Project Overview

Indoor navigation systems have become essential tools for helping people find their way in complex indoor spaces like shopping malls, airports, hospitals, and university campuses. As cities grow and buildings become more complicated, the need for effective indoor navigation solutions is increasing (Faragher & Harle, 2015). Traditional navigation methods rely on static tools like signs, directory boards, and paper maps. While these can be helpful, they often lack the flexibility and real-time updates needed for today's fast-paced environments, leading to confusion, especially in busy or unfamiliar places. For example, directory boards may provide limited information, and static signs can fail to assist users once they move away from them.

Despite the continued use of static signage, it does not provide the interactivity and instant updates that are important in large buildings. Interactive indoor navigation systems that offer real-time positioning and route guidance can greatly improve the user experience by providing accurate directions tailored to individual needs (Keerthana et al., 2020). These systems can also include features for people with disabilities, such as voice guidance for visually impaired users. By giving audio instructions, the system helps these users navigate safely and effectively. Real-time updates allow users to adjust their routes based on current conditions, such as crowd sizes or obstacles.

User orientation is another crucial part of indoor navigation that influences how people interact with their surroundings. Good systems help users understand where they are and what is around them, which improves decision-making while navigating. By including features like augmented reality (AR) or visual markers on maps, these systems can enhance user orientation, making it easier for first-time visitors or those unfamiliar with the space to find their way. This support reduces anxiety and confusion, leading to a better navigation experience (Keerthana et al., 2020).

A key function of advanced indoor navigation systems is calculating the shortest path to a user's destination. Traditional navigation can be complicated, requiring users to decide the best route based on available signs or maps. By using algorithms to find the shortest path, indoor navigation systems can optimise routes based on factors like distance, estimated travel time, and potential barriers. These algorithms, such as Dijkstra's and A*, not only make navigation more efficient but also improve overall satisfaction by reducing travel time and making it easier for users to navigate large and complex spaces.

The main challenge for indoor navigation is the lack of reliable GPS signals. GPS often fails indoors because walls and other structures block the signals, making traditional GPS methods ineffective (Brena et al., 2017). Recently, alternative solutions like Wi-Fi fingerprinting, geomagnetic sensing, and Bluetooth Low Energy (BLE) beacons have been explored as possible indoor positioning technologies. However, each of these methods has limitations. For example, Wi-Fi fingerprinting can be unreliable due to signal fluctuations, while geomagnetic sensing can be affected by changes in the environment.

To tackle these challenges, this project aims to develop a strong navigation system that provides real-time indoor positioning, dynamic route calculation, user orientation support, and accessibility through multilingual voice navigation. By combining different positioning technologies, our system seeks to improve accuracy and reliability. The voice guidance feature makes the system more inclusive, catering to visually impaired users and those who prefer hands-free navigation. In addition to addressing the challenges of indoor navigation, our project employs a range of technologies and tools to ensure effective implementation, such as Java for the frontend, Python for the backend, and Flask as the backend framework. We also utilise MapBox and MazeMap for mapping, SQLite for database management, and Bluetooth Low Energy (BLE) beacons for real-time positioning. Additionally, our project is designed to be scalable and adaptable, ensuring flexibility for use in various types of buildings and helping users reach their destinations efficiently.

1.2 Report Overview

In this report, we will begin by providing a comprehensive overview of the project's background, followed by a detailed literature review to establish the foundational context and highlight relevant work in the field. This will offer insight into the motivations behind the project and the gaps it aims to address. With a clear understanding of the project's objectives, we will then assess the project's outcomes by examining how the specified requirements have been fulfilled. This assessment will include an analysis of the alignment between the planned objectives and the actual results achieved. Additionally, we will explore any modifications made to the initial requirements and provide justifications for these changes. This thorough review will set the stage for understanding the progression from conceptualization to implementation.

Following this, we will delve into the methodology employed to accomplish the project's goals and introduce the overall system architecture, outlining its key components and their interconnections. This will provide readers with a clear understanding of the strategic approach taken to achieve the desired functionality. We will then present the final product in the software deliverables section, offering a detailed analysis of its quality across various dimensions. This section will lead into a critical evaluation of the project in the Software and Project Critique section, where we will reflect on the strengths, limitations, and challenges encountered. The discussion will be brought to a close with a comprehensive conclusion, summarising key insights, and highlighting potential directions for future work.

2. Project Background

In this section, we explore the technological landscape of indoor navigation, including the background and rationale behind our project, as well as an evaluation of related work that has influenced its implementation.

2.1 Background

2.1.1 The Evolution of Indoor Navigation Technologies Using Smartphones

Traditional methods for indoor navigation, such as static signage and paper maps, are inadequate in the context of modern multi-level buildings. With the rise of smartphones equipped with sensors like compasses, gyroscopes, and Bluetooth, indoor navigation systems have become more advanced (Li et al., 2015). By utilising these sensors, we can now offer real-time positioning, which tracks users' movement dynamically and helps them navigate complex indoor environments (Brena et al., 2017). Bluetooth technology, in particular, has emerged as a key enabler of accurate and responsive indoor navigation.

2.1.2 Transition to Bluetooth-Based Real-Time Positioning

Most indoor navigation systems rely on directory boards or static maps that become irrelevant once the user moves away from them. In contrast, real-time positioning technology leverages mobile devices and Bluetooth beacons to offer continuous, updated guidance. The flexibility of Bluetooth beacons allows for dynamic mapping, and their widespread compatibility makes them an ideal choice for indoor navigation (Lie et al., 2020). By integrating Bluetooth positioning, users can experience a seamless, real-time navigation experience that adapts as they move.

2.2 Rationale

This section discusses the motivations behind the choice of real-time positioning, orientation tracking, and shortest route calculation in our indoor navigation system.

2.2.1 Real-Time Positioning

Real-time positioning is a key feature of indoor navigation systems. By continuously tracking the user's location, the system can provide up-to-date navigation guidance. Our system employs Bluetooth beacons to estimate the user's location in real-time, updating their position on a 2D map interface. The map dynamically adjusts based on the user's movements, ensuring that users can accurately track their current position within the building. This feature is crucial for dynamic environments where static maps become obsolete.

2.2.2 Orientation

Orientation plays a crucial role in helping users understand which direction they are facing and how to follow the calculated route. The system uses a combination of device sensors, such as the geomagnetic sensor, to detect the user's heading. This data allows the system to adjust the map interface and provide clear visual and verbal instructions, ensuring users are oriented correctly while navigating. Incorporating orientation information significantly improves navigation accuracy, offering personalised guidance that considers not just where the user is, but also where they are looking (Gonzalez et al., 2020).

2.2.3 Shortest Route

In addition to real-time positioning, the system calculates the shortest route between the user's current location and their desired destination. By applying pathfinding algorithms such as Dijkstra's or A* (Hart et al., 1968), the system efficiently determines the optimal route. This ensures that users are guided through the quickest and most efficient paths, avoiding unnecessary detours.

2.3 Literature Review

The literature review section examines existing research and methodologies relevant to indoor positioning and navigation systems. It highlights the strengths and weaknesses of various techniques, providing context for our approach in developing a more effective indoor navigation solution.

2.3.1 Indoor Positioning Techniques

Accurate indoor positioning is crucial for effective navigation in complex indoor environments, and various techniques have been developed to address the unique challenges of indoor localization.

2.3.1.1 Wi-Fi Fingerprinting

Wi-Fi fingerprinting utilises existing Wi-Fi infrastructure to approximate the location of a device indoors by analysing signal strength measurements (RSSI) from nearby access points. This technique is commonly used due to its ability to leverage pre-existing networks, making it a cost-effective solution for large spaces (Pasricha, 2020). However, our experimental findings identified two major limitations with Wi-Fi fingerprinting. First, since we do not have control over Wi-Fi access points, we cannot optimise or guarantee the consistency of signal distribution, leading to variable accuracy. Second, Wi-Fi RSSI signals fluctuate frequently due to environmental changes, user movement, and network traffic, which impacts the reliability and precision of the model. These factors resulted in fluctuating signal strengths that affected the system's accuracy and made Wi-Fi a less viable option for stable indoor positioning.

2.3.1.2 Bluetooth Beacons

Bluetooth beacons, particularly Bluetooth Low Energy (BLE) beacons, offer an alternative by transmitting periodic signals that can be detected by nearby devices for proximity-based location tracking (Pasricha, 2020). BLE beacons are cost-effective, easy to deploy, and cover a signal range of up to 10 metres. Through our work, we found that BLE beacons address the primary issues encountered with Wi-Fi. Unlike Wi-Fi, BLE beacons do not rely on external infrastructure, giving us complete control over beacon placement and signal distribution. Additionally, the RSSI signals from BLE beacons were observed to be stable, significantly reducing inaccuracies. The signals from BLE beacons were also distinct and consistent, which enhanced the model's ability to differentiate between locations, ensuring a high degree of accuracy and reliability for indoor positioning.

2.3.2 Shortest Path Implementation

The optimal route calculation is essential in indoor navigation systems, particularly when users need to reach a destination quickly in environments with multiple pathways and variable traversal costs.

2.3.2.1 Dijkstra's Algorithm

Dijkstra's algorithm is one of the most popular techniques for finding the shortest path in a weighted graph. It operates by exploring all possible paths from a starting point to each node, iteratively selecting the least costly path until it reaches the destination (Makariye, 2017). This algorithm is highly suitable for indoor navigation, as indoor environments typically have weighted paths (e.g., hallways, escalators) that may require different traversal times or costs. However, Dijkstra's algorithm can be computationally intensive for large graphs due to its exhaustive search of all possible routes.

2.3.2.2 A Algorithm*

A* is another widely used pathfinding algorithm that enhances Dijkstra's approach by incorporating a heuristic function to estimate the cost of reaching the destination (Hart et al., 1968). This heuristic allows A* to optimise its search, making it faster and more efficient than Dijkstra in many cases. The choice of heuristic, however, can greatly affect the performance and accuracy of A*, which makes it less predictable in indoor environments with complex layouts and variable path weights.

2.3.2.3 Breadth-First Search (BFS)

BFS is a simpler algorithm that explores all nodes at the present depth level before moving to the next. While effective for finding unweighted shortest paths, BFS does not handle weighted graphs effectively, which is a significant drawback in indoor navigation scenarios where varying traversal costs must be considered.

2.3.3 User Orientation

User orientation is critical in indoor navigation, as it enables the system to understand the user's facing direction and adjust the route guidance accordingly. Several approaches exist to determine and track user orientation in real time:

2.3.3.1 Device Geomagnetic Sensors

Many smartphones come equipped with a geomagnetic sensor, also known as a digital compass, which detects the Earth's magnetic field to determine the user's heading. This method provides a straightforward way to establish orientation and works well in environments with minimal magnetic interference (Brena et al., 2017).

2.3.3.2 Inertial Measurement Units (IMUs)

Inertial Measurement Units (IMUs) combine accelerometers, gyroscopes, and magnetometers to capture data on movement and rotation. By integrating this data, the system can calculate the device's orientation and adjust guidance as the user moves (Ta et al., 2018). IMUs are highly effective for tracking orientation in real-time and are less susceptible to interference, making them a suitable choice for complex indoor environments.

2.3.3.3 Visual-Inertial Odometry (VIO)

Another advanced method, Visual-Inertial Odometry (VIO), combines visual data from the device's camera with inertial measurements to improve orientation accuracy. VIO can be particularly useful in indoor navigation systems as it accounts for visual cues from the environment, making it highly robust. However, this approach requires more processing power and can be affected by changes in lighting or visual clutter in the environment (Ta et al., 2018).

2.4 Conclusion

In light of the challenges with Wi-Fi fingerprinting, we selected BLE beacons for indoor positioning. BLE provided a more stable, controllable, and reliable RSSI signal, free from the external dependencies and fluctuations experienced with Wi-Fi. This allowed us to create a more accurate model and enhance the system's reliability, supporting seamless indoor navigation.

In our case, we selected Dijkstra's algorithm for shortest path calculation. While A* could potentially be faster with an effective heuristic, our testing environment showed that Dijkstra's algorithm provided consistent results and accurately reflected traversal costs. Since indoor paths often involve a variety of obstacles and traversal times, Dijkstra's approach was optimal for accurately calculating the shortest paths without requiring additional heuristics or assumptions about the environment.

For this project, we rely solely on the geomagnetic sensor integrated into most smartphones to determine the user's orientation. This sensor detects the Earth's magnetic field, enabling the system to calculate the user's heading direction in real-time. By focusing exclusively on the geomagnetic sensor, we ensure a straightforward and reliable orientation solution without needing complex sensor fusion techniques or additional hardware.

3. Outcomes

In this section, we will discuss the features we implemented and how they relate to our initial project requirements. We will begin by breaking down the app into small features, explaining how each interacts with users. Next, we will demonstrate how each feature works with screenshots of our application. Then, we'll link these features back to our RTM (Requirements Traceability Matrix) to see which requirements have been met and how they are fulfilled. For any requirements not met or modified, we'll provide justifications. Following that, we'll examine the limitations and shortcomings of the application as a whole. Lastly, we'll explore possible future work to build upon what we have achieved so far.

3.1 What Has Been Implemented

This section provides a detailed breakdown of the features and capabilities our final product offers to users, organised according to the app's initial design objectives. We begin by describing each implemented feature, explaining how it functions and how users will interact with it in real-time. This section aims to illustrate the app's utility by connecting each feature back to our original project requirements and discussing the benefits they bring to the overall user experience.

3.1.1 Automatic Indoor Position Detection (*Localization*)

BLE (Bluetooth Low Energy) beacons, chosen for their efficiency, low power consumption, and capability to emit signals at relatively low frequencies. Each beacon is strategically placed to cover optimal intervals—approximately 6 to 8 metres—so that overlapping signal ranges enable continuous tracking. During the initial setup, we created a "fingerprint" of each point by mapping signal strength levels from various beacons. When a user launches the app, Bluetooth signals from surrounding beacons are collected in real-time, and our AI-powered localization algorithm predicts the user's exact position on the map almost instantly.

As users move, signal strengths from nearby beacons fluctuate depending on their proximity, which triggers our backend to predict the user's updated location based on these changes. These dynamic predictions are displayed on a 2D map, where a direction-indicating arrow reflects their movement. This real-time tracking eliminates the need for scanning markers or QR codes, offering seamless and immediate location detection.

3.1.2 Shortest Path Calculation

Our application calculates the shortest possible route from the user's starting point to their selected destination. We incorporated Dijkstra's algorithm to generate optimal paths, with considerations for both stairs and lifts. In cases where a lift provides a shorter walking distance than stairs, the algorithm factors in these differences to ensure the shortest overall route is selected. This feature is particularly valuable in multi-floor environments, where transitioning floors efficiently can reduce travel time. Visual representation of these routes

ensures that users can easily follow the displayed path, with intuitive indicators for floor changes where applicable.

3.1.3 Dynamic Rerouting

To ensure users remain on track, our application dynamically reroutes if the user deviates from the recommended path. This process involves recalculating the route from the user's current position, turning it into the new starting point for the shortest path calculation. Our system continuously monitors the user's location based on signal updates, adjusting the path in real-time to guide users along the most efficient route to their destination. This feature enhances flexibility, offering real-time adaptability and reducing any potential frustration if a wrong turn is taken.

3.1.4 Student Account Integration

Our application is designed to streamline navigation for students by integrating with their accounts to access scheduled class information. Instead of manually entering classroom numbers, users can log in to view their class schedule by date and time. They can select their classroom directly from this schedule, triggering the app to auto-navigate to the classroom location with a single click. This feature minimises input time and simplifies the navigation process, making it easy for students to reach classes without memorising room numbers.

3.1.5 Multi-Floor Navigation

Our application supports multi-floor navigation, allowing seamless transitions between different floors of a building. As users move to a new floor, the app automatically detects signal patterns indicative of the change and switches the 2D map view to the new floor layout. The current floor's route remains visible, while routes on other floors are shown only when users reach the relevant level. This feature reduces visual clutter and ensures users can easily follow the path without confusion, making multi-floor navigation intuitive and straightforward.

3.1.6 Voice Navigation with Multi-Lingual Support

To enhance accessibility and ease of use, our app includes voice navigation, which users can toggle on or off as needed. Recognizing Malaysia's multicultural environment, we implemented support for five languages: English, Chinese, French, Malay, and Indonesian. Once a language is selected, both the UI and voice prompts automatically adapt to the choice, providing a seamless experience. For each step of the route, updated voice directions correspond with real-time changes in the user's position or orientation, ensuring users receive accurate verbal guidance throughout their journey.

3.1.7 ETA And Distance Visualisation

To keep users informed of their remaining travel time and distance, we calculate and display an estimated arrival time (ETA) and distance to the destination. The remaining distance is based on the length of the shortest path, while the ETA is dynamically updated by dividing

this distance by the user's estimated walking speed. Initially set at an average speed of 1 m/s, the app adjusts this estimate as users move, refining it based on calculated speeds during each localization update. This feature helps users manage their time effectively, adding a practical layer to the navigation experience.

3.1.8 Orientation Sensing

Using the gyroscope sensor on users' devices, our app determines their orientation to provide accurate, real-time directional guidance. As users rotate or turn, even by small degrees, the app senses these changes, allowing the directional arrow and map to align with the user's current facing direction. This orientation-aware navigation creates an intuitive experience where the map "moves" with the user, providing a highly responsive and realistic guide that significantly improves overall navigation accuracy and user satisfaction.

3.2 Product Delivered

This section provides a detailed overview of the core functionalities delivered in our final product. Our application stands out from traditional indoor navigation systems through its automatic position detection and hands-free navigation experience, eliminating the need for physical markers or scans to initiate location tracking.

3.2.1 Automatic Indoor Position Detection (Localization)



Diagram 3.2.1: User Launches the Application

Unlike the existing indoor navigation applications, users no longer need to scan a marker to indicate their location. Our application automatically detects their location upon launching. The arrow pointer indicates the user's orientation; wherever the arrow is pointing represents the direction in which the user is facing.

3.2.2 Shortest Path Calculation / Eta and Distance Visualisation / Voice Navigation with Multi-Lingual Support

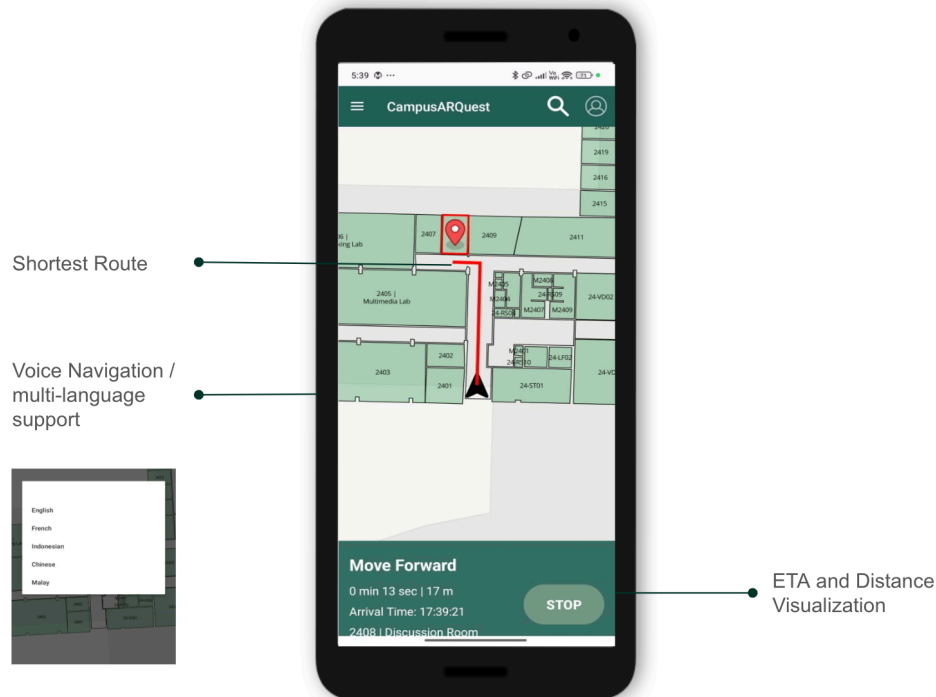


Diagram 3.2.2: Specific Route Displayed

The application provides an efficient route calculation, displaying the shortest path from the user's current location to their destination on a 2D indoor map. The red path on the map in Diagram 3.2.2 visually guides the user to their selected location, with turn-by-turn navigation provided through a combination of visual cues and voice prompts.

In addition, users receive real-time information on the estimated time of arrival (ETA) and remaining distance to their destination, displayed below the map. This feature allows users to gauge their progress and plan their journey effectively. For enhanced accessibility, the application offers multilingual support, enabling users to select their preferred language for voice navigation. As shown in the language selection box on the left, users can choose from multiple languages, making navigation accessible to a diverse audience.

3.2.3 Student Account Integration

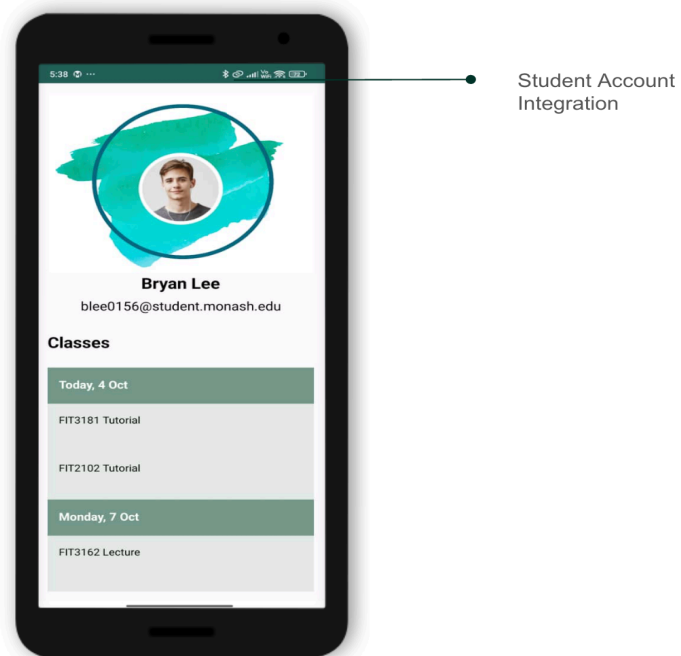


Diagram 3.2.3: User's Profile with Details of Classes

The interface presents a clear and organised schedule, displaying current and upcoming classes with their respective dates and course details. With a personalised profile view, students can effortlessly select a class from their schedule, triggering the app to auto-navigate them to the classroom location with a single click. This feature minimises manual input, enhances ease of navigation, and reduces the need to memorise room numbers, improving the overall user experience.

3.2.4 Orientation Sensing

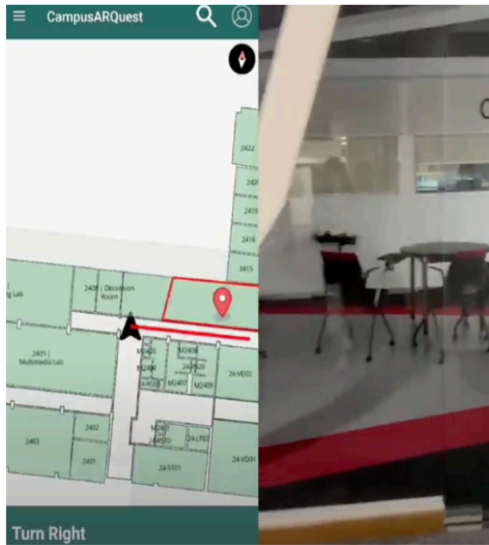


Diagram 3.2.4.1: Before User Turns Right

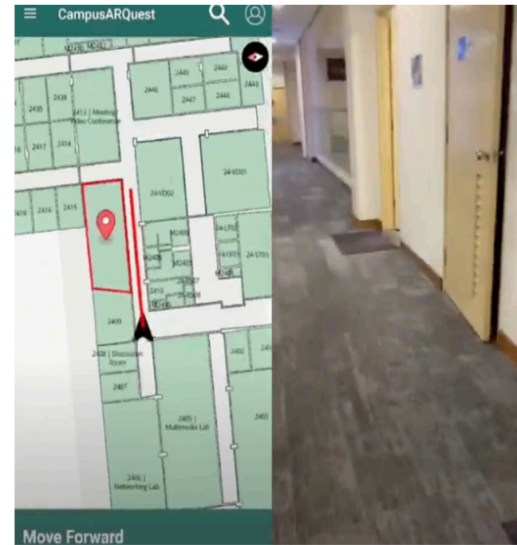


Diagram 3.2.4.2: After User Turned Right

The above diagram shows how the user's orientation will update the display of our 2D map. Initially, the user is guided to turn right. Once the user turns right, their orientation is updated, and the map rotates to match their latest orientation, ensuring that the arrow pointer is always pointing upward. Making sure the arrow is always pointing upwards means the map is always displaying the direction in which the user is facing. The directional guide will also be updated to direct the user to move forward.

3.2.5 Dynamic Rerouting

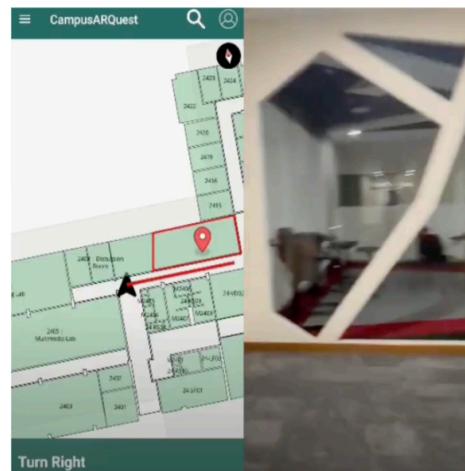


Diagram 3.2.5.1: Before User Turns Right

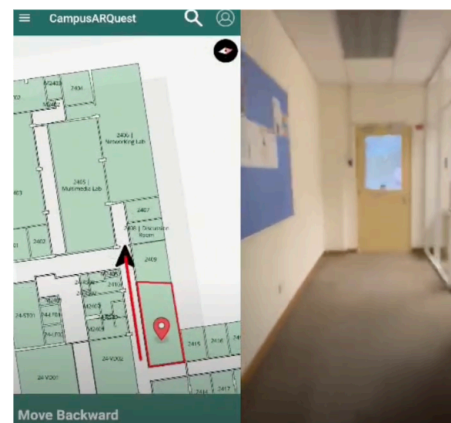


Diagram 3.2.5.2: User Turned Left Instead

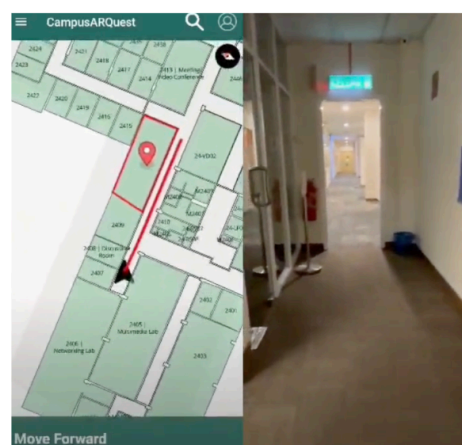


Diagram 3.2.5.3: Rerouting After User Continued with Wrong Path

When the user walks down the wrong path, as presented in Diagram 3.2.5.2 above, the app will detect that the user turned left instead of right, and will prompt them to move backward. If the user continues walking in the wrong direction, both the directional guide and the route will be updated to reflect the user's latest position.

3.2.5 Multi-Floor Navigation

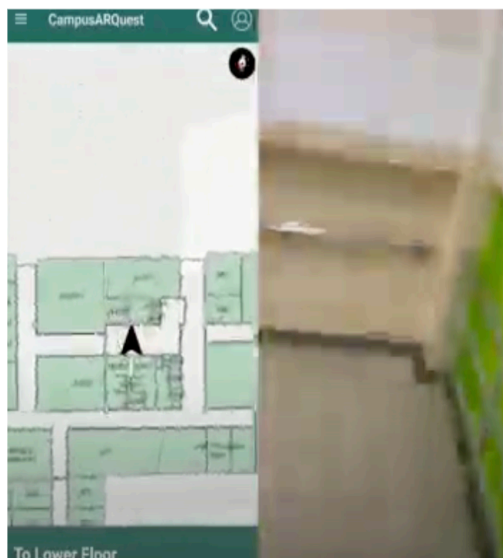


Diagram 3.2.6.1: User is in Floor 4

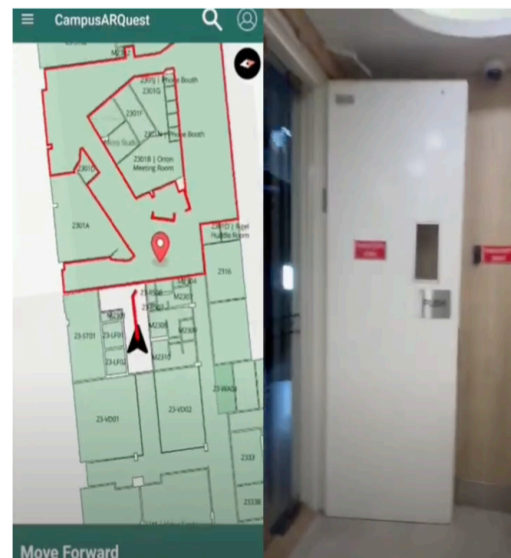
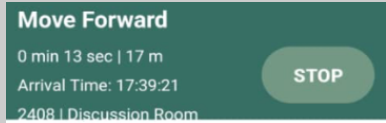


Diagram 3.2.6.1: User Reached Floor 3

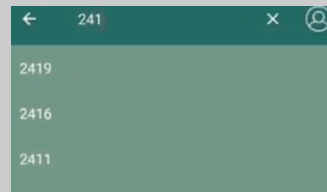
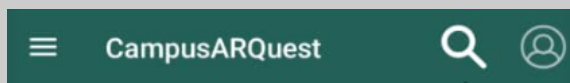
The system automatically guides users across different floors by dynamically updating the 2D map based on their location. When a user enters a new floor, the map seamlessly transitions to display the corresponding floor's layout, including the remaining route for that specific level. As illustrated in the diagrams, when the user moves from floor 4 to floor 3, the entire map updates to render the new floor along with the adjusted path, ensuring that users can easily follow their route without needing to manually switch maps or reorient themselves. This feature enhances navigation efficiency and user experience across multi-level environments.

3.3 How Project Requirements Are Met

This section provides a comprehensive overview of how our indoor navigation application meets the original requirements, derived from the Requirements Traceability Matrix (RTM). Each requirement is analysed, with a clear explanation of how it is fulfilled in our working application. Where requirements were modified or not met, the changes are highlighted, and justifications are provided in [section 3.4](#). This structured assessment demonstrates our commitment to delivering a reliable, user-centred application while navigating practical constraints and resource limitations.

ID	Description	How the requirements are met
FR1	Application will be functioning for any indoor area of the 2nd floor of building 6	This requirement was not met as we switched the scope to cover the 3rd and 4th floors of building 6 instead. However, we extended the scope from only covering one floor to multiple floors, allowing for greater usability across a larger area.  <pre> { "type": "Feature", "geometry": { "type": "Polygon", "coordinates": [[[101.60035706357044, 3.0647201599350025], [101.60035697958021, 3.064709487653373], [101.60035706357044, 3.0647201599350025]]] } } </pre> <i>Figure 3.3.1: Geojson Data of Building 6 Floor 3</i>
FR2	Application will display directions on screen to allow users to navigate indoor areas.	Directions are displayed on the formation bar at the bottom of the UI as the user begins navigation. This enhances the user experience by providing clear, on-screen guidance throughout their journey.  <i>Figure 3.3.2: Navigational Direction Displayed on Progress Bar</i>

FR3	The application will track the user's position using Wi-Fi signals and update the navigation direction in real time.	This requirement was not met as we opted to use BLE signals instead of Wi-Fi. Nevertheless, we successfully track the user's position and update navigation directions in real-time using BLE technology, offering an effective alternative.
FR4	The application will have audio cues that direct users with visual impairment.	Audio cues serve as the primary guidance alongside text for all users. Additionally, users have the option to toggle audio cues on or off, making the application accessible for those who may not require auditory guidance.
FR5	The application will be developed with English as its only choice of language.	We expanded the scope of this requirement; we programmed the application to support five languages: English, Chinese, French, Malay, and Indonesian. This broadens accessibility and enhances the user experience for a diverse audience. <pre> <string name="turn_left">向左转</string> <string name="turn_right">向右转</string> <string name="walk_straight">直走</string> <string name="go_up">上一层楼</string> <string name="go_down">下一层楼</string> <string name="walk_backward">向后走</string> <string name="complete">您已到达目的地。</string> </pre> <i>Figure 3.3.3: String Resource File for Different Language</i>
FR6	The application will show the user the shortest route from initial position to destination at that particular time.	This feature was successfully implemented using Dijkstra's algorithm on a graph where possible destinations are vertices and paths are edges. Distances are translated into weights, ensuring users receive the shortest route efficiently. <pre> def get_nearest_neighbour(self, s_vertex, floor): nearest_neighbour = (math.inf, None) for vertex in self.vertices.values(): distance = self.compute_gcd(vertex, s_vertex) if vertex.z == floor: nearest_neighbour = min(nearest_neighbour, (distance, vertex), key = lambda tuple: tuple[0]) return nearest_neighbour </pre> <i>Figure 3.3.4: Code for Getting Nearest Vertex Based on Weight</i>
FR7	A 2D and 3D map will be developed for indoor navigation.	This requirement was not met; we focused solely on developing a 2D map for navigation. While 3D navigation could enhance user experience, resource

		constraints limited our implementation to a 2D interface.
FR8	The application must request and obtain permission to access the device's camera.	Users will be prompted to grant camera and bluetooth access during their first time of launching the app.
FR9	The application must allow users to select a POI from a list.	Users can choose their desired destination by typing in the location in a search bar. The application provides a list of recommended destinations, streamlining the process of finding and selecting points of interest (POIs).  <i>Figure 3.3.5: List of recommended POIs</i>
NFR 1	The application will be developed with a user interface that conforms to industry-wide practice for consistency and standards, as well as ISO/IEC JTC1/SC 35.	During development, we adhered to established guidelines, positioning a hamburger menu in the upper left corner to ensure a familiar navigation structure. We involved users in the design process through usability testing and feedback sessions. For instance, we create prototypes and conduct user testing to identify pain points in the navigation flow or information architecture. We decided to use universally recognized icons (like a magnifying glass for search) along with text labels to enhance understanding. This can ensure that the icons are intuitive and help users navigate the interface without confusion.  <i>Figure 3.3.6: Toolbar with Universally Recognized Icons</i>
NFR 2	The application must render AR elements such as virtual arrows to be displayed on top of the	This requirement has not been satisfied. Further explanation will be in the following section.

	real-world view.	
NFR 3	The application must be responsive and performant, with load times under 10 seconds.	To increase the performance of our application, we utilise multiple threads to parallelize the tasks properly. Some threads will handle rendering of ui while some handles detection of BLE signals. The load time differs with each prompt, affected by the factors of web traffic, wifi strength, and surrounding obstructions. Nevertheless, the longest load time we had was around 7 seconds.
NFR 4	The application must provide big enough text sizes.	Text sizes are implemented based on WCAG (Web Content Accessibility Guidelines). For instance, we maintained a size difference of at least 1.3 times between body text and headers, enhancing clarity for users.
NFR 5	The application must ensure high contrast between text and background.	This requirement was addressed during the FYP1 design phase, where we ensured strong contrast between text and backgrounds. A dark green toolbar with bold white text was implemented to enhance visibility and user engagement.
NFR 6	The application must provide clear and concise instructions.	We implemented a bar to indicate directional guide in bolded text, alongside with distance left, and estimated time of arrival. The destination is prominently displayed, and users can stop navigation by clicking a bright, round "STOP" button, ensuring intuitive usability.
NFR 7	The application must process wifi signals and update the user's position within 10 seconds.	The user's position is detected within 10 seconds; however, we decided to use bluetooth instead of Wi-Fi signals.

Table 3.3.1: Requirement Traceability Matrix vs How the Requirements are Met

In summary, the requirements for our indoor navigation application were met to a significant degree, ensuring a functional and accessible tool for indoor navigation across multiple floors. Modifications were made thoughtfully, balancing user needs, technical feasibility, and project scope. Although a few requirements were not fully achieved, careful consideration was given to providing effective alternatives or enhancements, as detailed in section 3.4. Through adherence to standards and an emphasis on usability, we successfully delivered an

application that meets most critical requirements and prioritises an improved user experience.

3.4 Justification of Decisions Made

In the development of our indoor navigation application, a number of critical decisions were made regarding the project requirements outlined in the Requirements Traceability Matrix (RTM). As we progressed through the project, we encountered various challenges and opportunities that led to modifications in our original requirements. This section elaborates on the justifications for these modifications, explaining how they align with the overall goals of the project and contribute to delivering a functional and user-friendly application.

ID	Description	Justification on Modifications
FR1	Application will be functioning for any indoor area of the 2nd floor of building 6	The decision to extend the application's scope to include the 4th floor of Building 6 was influenced by several factors. First, the 4th floor is affiliated with the School of IT, which provides more opportunities for practical application testing. After we got our app running in a small corridor, our supervisor advised us to extend the map to cover multiple floors. This can make our application more impotent by solving more existing problems in indoor navigation apps. Hence, we decided to extend our scope to cover 2 floors in the end. By including two floors, we increased the app's relevance and utility for a larger audience.
FR3	The application will track the user's position using Wi-Fi signals and update the navigation direction in real time.	After collecting data from WiFi signals, we realised that WiFi fingerprinting was not accurate for user localization. Although research has shown WiFi signals to be more effective than Bluetooth signals in some cases, our WiFi was set up by school authorities for general use rather than precise location tracking. Variability in Wi-Fi signals, influenced by environmental factors, further compromised accuracy. Additionally, WiFi signals vary greatly from day to day due to factors like weather and power fluctuations. More of this will be explained under Section 4.1.2. Following our supervisor's advice, we pivoted to using BLE beacons instead. We bought BLE beacons with a transmitter range of 100m and broadcasting frequency of 500ms.

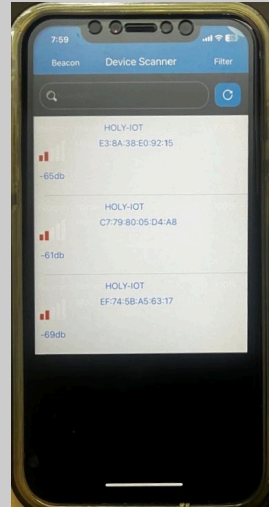


Figure 3.4.1: Detecting BLE Signals

However, we noticed that the broadcasting frequency and transmitting range was not as good as advertised. To obtain vast contrast in BLE signals, we decided to place beacons at every 8m. By trial and error, we derived optimal placements for the BLE beacons. These Bluetooth signals effectively tracked the user's position and updated navigation directions in real time.



Figure 3.4.2: Red Crosses Represents Placement of Beacons

FR5

The application will be developed with English as its only choice of language.

While developing the application, we discovered an API called Text-to-Speech (TTS) in Android that enables voice navigation. After implementing it, we thought, wouldn't it be nice to go further by offering voice navigation in multiple languages, considering Malaysia's multicultural and multilingual society. Therefore, we decided to include additional language options to enhance the user experience. This modification reflects our commitment to inclusivity in application design.

FR7/ NFR 2	A 2D and 3D map will be developed for indoor navigation. / The application must render AR elements such as virtual arrows to be displayed on top of the real-world view.	After realising the android device from our campus has an incompatible android version, we purchased a Samsung device for the project at a cost of 700 ringgit. Unfortunately, we overlooked the fact that this phone lacked hardware support for essential features like AR rendering. Our initial plan to use ARCore was thwarted because the device wasn't compatible with the API, which blocked our intended AR functionalities. We also experimented with alternative AR tools, such as Vuforia, but encountered the same issue due to the device's limited hardware capabilities. Even if we had succeeded in implementing AR, the rendering would have demanded too much processing power, likely degrading the app's performance. Ultimately, we chose to exclude AR features. Rather than compromising navigation performance to include AR, we chose to focus on optimising the 2D map for better performance and decided not to pursue 3D navigation.
------------------	--	--

Table 3.4.1: Requirements That Are Modified and Their Justifications

In conclusion, the modifications made to our project requirements were driven by practical considerations and a commitment to enhancing user experience. Each decision reflects a careful evaluation of the challenges faced during development, alongside a strategic focus on delivering a robust and effective indoor navigation application. By embracing flexibility and prioritising user needs, we have created a solution that not only meets but exceeds the initial requirements, ultimately providing a valuable tool for indoor navigation. This iterative approach has equipped us with insights and experience that will inform future projects and improve our overall development processes.

3.5 Limitations of Project Outcomes

Despite achieving our primary objectives, our project faced certain limitations that impacted its overall scope and effectiveness. This section explores these constraints, focusing on the small coverage of the campus map, response time delays, high server load during peak hours, and the lack of augmented reality (AR) features. Each of these limitations presents challenges that hinder the user experience and limit the project's potential to scale and improve. Addressing these issues in future iterations could significantly enhance the app's reliability, user satisfaction, and adaptability to real-world conditions. The following subsections provide an in-depth examination of each limitation, its implications, and potential solutions.

3.5.1 Small coverage of campus map

Our project currently encompasses only two floors of the campus building due to a limited number of beacons available for deployment. Initially, we aimed to minimise risk by starting with a small number of beacons and gradually mapping the two floors, ultimately using a total of 20 beacons. This strategic approach was necessary to validate our method without committing extensive resources upfront. However, because of time and budget constraints, we could not purchase additional beacons after that, leading to our decision to limit the project's scope to these two floors.

The implications of this limitation are significant. With only two floors covered, students may find themselves navigating areas that are not included in our mapping, creating potential frustration and reducing the effectiveness of the navigation tool. The project's efficacy hinges on its ability to provide comprehensive coverage, as users will expect reliable guidance throughout the entire campus. If they frequently encounter "unknown areas," it could diminish their trust in the application.

3.5.2 Long response time in updating user's position

The response time delay when a user moves to a new location is influenced by several factors. During our testing, we recorded a maximum response time of 7 seconds, particularly in crowded corridors where signal reception was compromised. While the average response time hovered around 3 seconds, this is still suboptimal for a navigation application where quick updates are essential.

This delay is particularly problematic when users are moving quickly—such as when they are rushing to a class—because the application may not be able to update their position in real-time. The core issue stems from the low emission frequency of Bluetooth signals from the beacons, which have relatively long resting periods between signals. If a user moves to a new location during this resting phase, no updated feedback is sent to the application, leaving it unaware of the user's new position.

To illustrate, if a student runs from one end of a corridor to another, they may find that the application still shows them at their previous location for a few seconds, causing confusion and undermining their navigation experience. Addressing this limitation would require exploring options to increase the signal emission frequency or employing alternative localization methods that can provide faster updates.

3.5.3 Longer Loading of Route During High Volume of Traffic

As the number of users querying the server increases, we observed a noticeable slowdown in response times. With many users accessing the application simultaneously, the route rendering process can become sluggish, detracting from the overall user experience. For example, during peak hours when students are transitioning between classes, the server may struggle to keep up with the volume of requests, leading to longer loading times for routes.

Moreover, if the same user frequently changes their destination, such as reconsidering their route mid-journey, this repeated querying can exacerbate the server's response delays. The cumulative effect of high traffic not only frustrates users but can also lead to decreased engagement with the app, as students may seek alternative methods to navigate the campus.

To mitigate this issue, we could explore options for optimising server response times, such as implementing load balancing to distribute traffic more evenly or employing caching strategies to reduce the number of queries processed by the server. For instance, frequently requested routes could be cached to expedite response times for those specific requests.

3.5.4 No AR Implementation For Camera View

Although our application provides robust 2D navigation features, we have yet to integrate 3D navigation or Augmented Reality (AR) functionalities. This limitation arises from the dependency loop between hardware capabilities and software requirements, as outlined in [Table 3.4](#). Currently, our application does not support AR functionalities, preventing users from receiving real-time directional guidance when they switch to a camera view.

The lack of AR capabilities may limit the application's appeal to tech-savvy users who increasingly expect immersive features in navigation tools. For instance, students often utilise AR navigation in other contexts (such as in mobile games or other navigation apps) and may find the absence of this feature in our application a drawback.

Future development could focus on addressing this limitation by investing in compatible hardware or software solutions that facilitate AR integration. For example, we could explore partnerships with manufacturers of devices that support ARCore or similar platforms. This could open up opportunities for richer user interactions, such as overlaying directional arrows and information onto the camera feed, enhancing the navigational experience for

users. By addressing this gap, we could significantly increase user engagement and satisfaction.

3.6 Potential Improvements and Future Work

While our project successfully achieved its initial objectives, there are several key areas where improvements and future enhancements can be made to extend its functionality and efficiency. These potential improvements focus on expanding the project's scope, enhancing system performance, and providing users with a more refined and personalised experience. By addressing these areas, we can build on the project's current achievements and work towards delivering a more comprehensive and reliable solution for campus navigation.

3.6.1 Scaling up the map

Since our project had proven the methodology to be working among 2 floors, future work can extend the map to cover the entire building or even campus. However, our methodology used might not be suitable for a larger scale of map. Our current methodology—Bluetooth signal fingerprinting—has a significant drawback regarding time efficiency during the preparation phase. During the fingerprinting stage, data collection requires standing at each point for 2–3 minutes to gather sufficient Bluetooth signals. With 20 points along the corridors of a 32m x 33m floor, this process took longer than anticipated. Therefore, scaling up to a larger area would make the fingerprinting method less feasible.

For future implementations, investing in a triangulation method—such as using trilateration or multilateration—could streamline this process. For example, instead of gathering signals at individual points, we could leverage multiple beacons' signals simultaneously to calculate user positions. This would reduce the time spent on site while improving the overall efficiency of the mapping process.

3.6.2 Rent server that can afford higher traffic load

To enhance user experience during peak usage times, we recommend renting a more robust server capable of handling higher traffic volumes. For instance, moving from a standard shared hosting plan to a dedicated or cloud-based server—like those offered by AWS or Azure—would allow for better scalability. With cloud services, we could implement auto-scaling features that automatically adjust server capacity based on real-time user demand.

An example of this in action would be during orientation week at the university, when new students are using the app extensively. A cloud server could accommodate the increased traffic, ensuring that response times for route rendering remain swift and reliable, thus maintaining a positive user experience.

3.6.3 Implementation of AR feature for camera view

Exploring the implementation of augmented reality (AR) features for the camera view remains a priority for future developments. To achieve this, we would need to ensure we have access to multiple android devices that possess the necessary hardware capabilities and compatibility with reliable AR frameworks, such as ARCore or Vuforia.

For example, we could enable users to point their camera at a location and see an overlay of directional arrows guiding them to their destination, enhancing navigation clarity. This AR feature could be particularly beneficial in complex areas, like multi-level buildings or when navigating through crowded spaces. To facilitate this, we could focus on devices like the Google Pixel series, known for their superior AR capabilities.

3.6.4 Expanding Beacon Coverage and Enhancing Localization Accuracy

To improve localization accuracy, we plan to invest in additional Bluetooth beacons. For instance, deploying 50 beacons instead of the current 20 could allow for a denser fingerprinting grid, resulting in more precise localization data. By conducting fingerprinting at closer intervals (e.g., every 3 metre instead of every 8 metres), we could create a more detailed map of signal strength variations, allowing for better location accuracy.

Additionally, we could explore using advanced algorithms, such as Kalman filters, to refine location estimates based on the multiple signals received from the beacons. This approach could help smooth out inaccuracies caused by environmental factors like signal interference or obstruction.

3.6.5 Improving Response Time Through Beacon Frequency Adjustments

Investigating ways to adjust the emitting frequency of Bluetooth signals from the beacons could lead to faster response times. For example, if we could decrease the resting period between signal emissions from the current settings 5 seconds to 1 second, we would receive location updates more frequently.

This improvement would be particularly beneficial in scenarios where users are running or moving quickly. In such cases, a more frequent update could provide real-time navigation feedback, ensuring that users receive timely and accurate directional cues.

3.6.6 User Feedback Mechanisms for Continuous Improvement

Establishing a feedback mechanism within the application can help us gather valuable insights from users regarding their navigation experience. For instance, we could implement a simple rating system at the end of a navigation session, asking users to rate the clarity of directions or the overall experience.

Furthermore, we could encourage users to report specific issues directly through the app. For example, if a user encounters a dead zone where the app fails to provide accurate directions, they could easily submit this information. Analysing this feedback would allow us to identify areas for improvement and prioritise updates, ensuring that the application evolves in line with user needs and preferences. This user-centred approach could significantly enhance overall satisfaction and usability.

3.6.7 Allow users to have more customization

In addition to integrating student accounts that provide information about class schedules and locations, we can enhance user experience by allowing greater customization options. One significant improvement would be enabling users to add their own events directly within the app.

For instance, students could create personalised entries for extracurricular activities, such as club meetings, sports practices, or study groups. Each user would have the ability to input the event's time and location, tailoring their navigation experience to their individual needs.

3.6.8 Event Notifications and Navigation Integration

To ensure users stay on track, we can implement a notification system that alerts them five minutes before their scheduled events. For example, if a student has a soccer practice at 3:00 PM, they would receive a reminder at 2:55 PM. This notification could include a prompt that, when clicked, instantly opens the navigation feature within the app, guiding them to the designated location.

By incorporating these custom features, we can provide a more tailored navigation experience that aligns with each student's unique schedule. This would not only enhance user engagement but also help students manage their time more effectively, ensuring they arrive at their commitments punctually.

4. Methodology

This section outlines the key components of our system architecture, provides a detailed breakdown of activities involved in each key component and states the software & hardware tools that are used in the project.

4.1 System Architecture

4.1.1 Overview

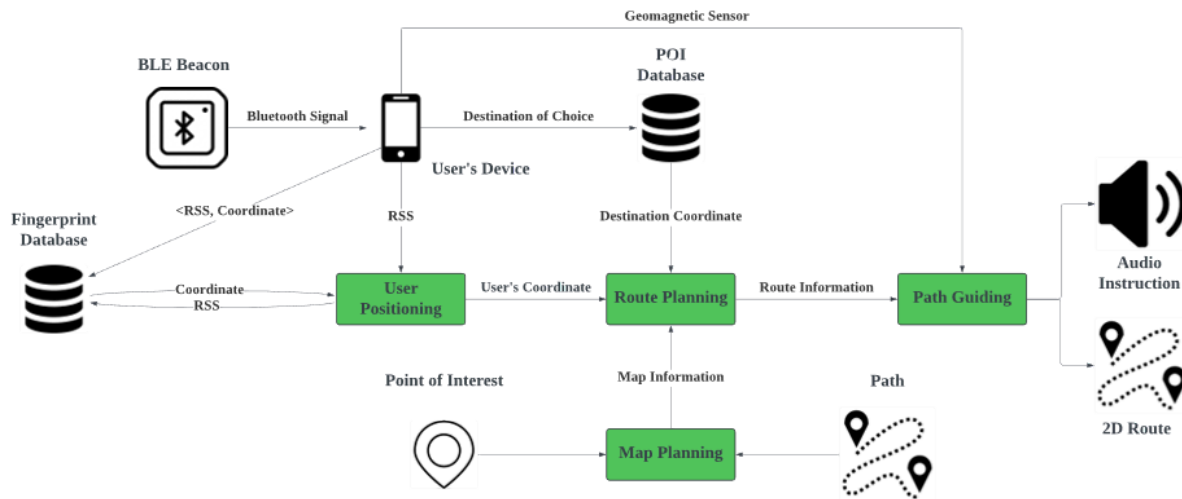


Figure 4.1.1.1: Architecture of Our Indoor Navigation System

Our system involves four distinct components: Map Planning, User Positioning, Route Planning and Path Guiding as shown on Figure 5.1.1. The development phase starts off with the Map Planning component, this is where the physical representation of the campus such as the floor plans will be constructed into a 2-dimensional digital representation.

The User Positioning component involves two different phases: 1) The positioning/online phase will determine the user's position using the received signal strength (RSS) value from the BLE signal emitted to the user's device. 2) The training/offline phase will involve the population of the fingerprint database by collecting the RSS value from the BLE signals emitted by designated BLE beacons at various locations on the map.

During the Route Planning component, the base-map data and user position data will be parsed into a specific data structure and shortest path algorithms are utilised to obtain the shortest path from the user's position to the destination of choice.

In the Path Guiding component, the path data computed by the Route Planning component is used to construct a 2-dimensional path within the digital map representation. Additionally, geomagnetic sensor readings from the user's device will be used to determine the user's orientation. By combining the orientation data with the route information, we can calculate the expected direction based on the user's current context. The expected direction will be output in an audio-based format from the user's device to assist with the route navigation effectively.

4.1.2 Deviation

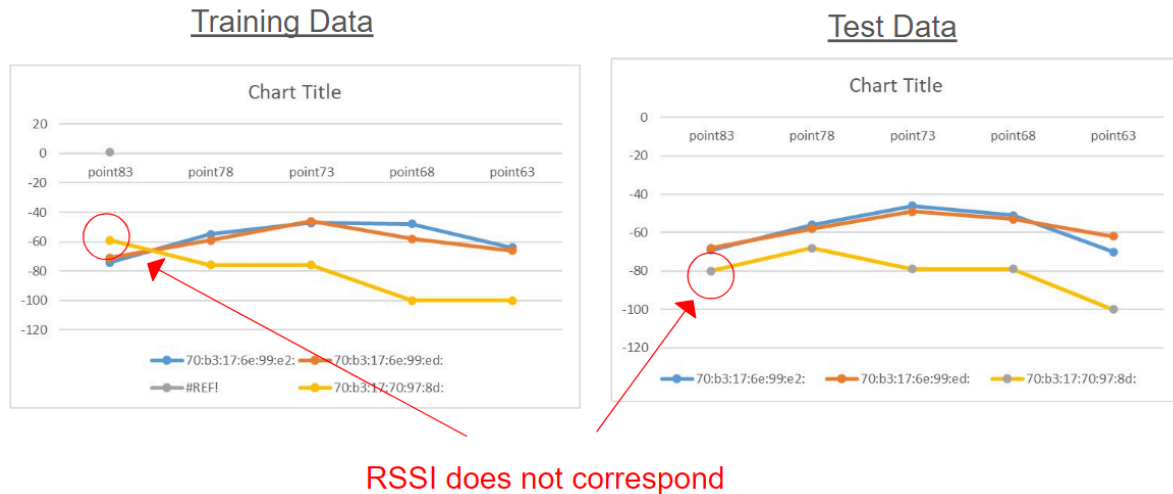


Figure 4.1.2.1: Limitation of WiFi Access Point

One major change in our system architecture from the project proposal is the choice of the indoor positioning technology. Initially, we decided to utilise the WiFi Access Point as our indoor positioning technology. However, we discovered limitations where RSS values collected from WiFi signals emitted by nearby access points are inconsistent with the expected characteristics which resultingly affected the accuracy of our User Positioning component. To resolve this problem, we pivot our indoor positioning technology to the BLE Beacon approach so that we can strategically position the BLE beacons to attain desired characteristics that can improve the accuracy of our User Positioning component.

4.2 Map Planning

4.2.1 Overview

To create a 2-dimensional digital representation of the map, we leveraged on the existing indoor navigation solution of the environment that we are mapping by designing a script that requests for GeoJSON data from the MazeMap API and filters the data to the specific floor and building that fits the scope of our project.

With the physical representation of the map created, we extended the acquired data by filtering it to a lightweight route map and labelling the objects of the GeoJSON data to ensure that the route map can be accurately parsed into the specific data structure used in the Route Planning component.

4.2.2 Deviation

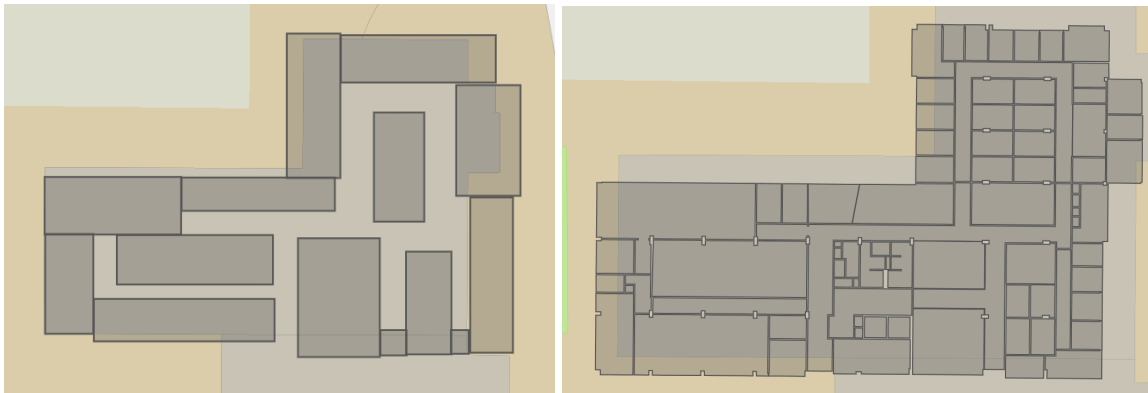


Figure 4.2.2.1: Comparison of Different Approaches in Attaining 2D Map Representation

In our project proposal, we suggested the use of a manual map design where we had to individually draw and label each object. We realised that this was a very time consuming process and discovered a way of automating the map planning process by designing a script that scrapes the data from the MazeMap API.

4.3 User Positioning

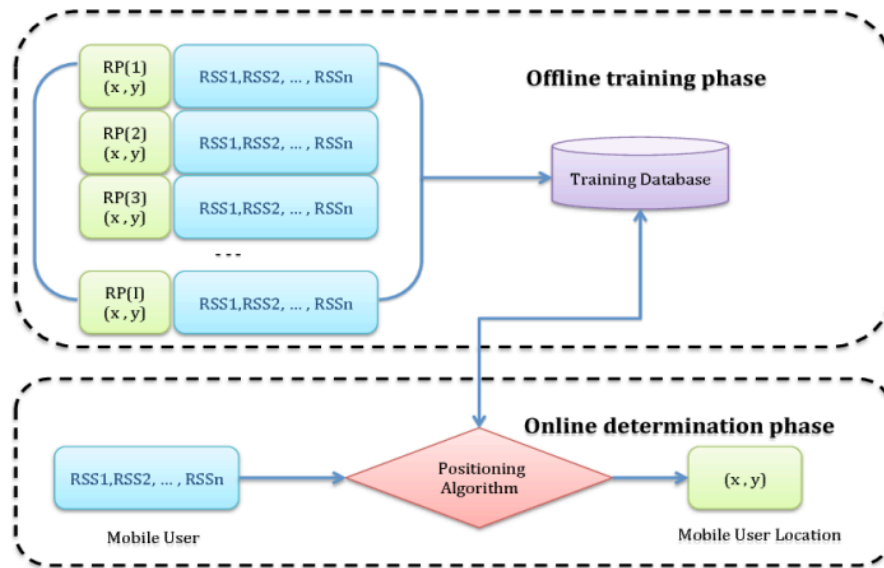


Figure 4.3.1: Overview of the Fingerprinting Technique (Zhao et al., 2018)

The chosen methodology of our User Positioning component is the fingerprinting technique because the environment covered by our project scope poses significant and common issues like signal interference. Therefore, the triangulation methodology is not ideal due to the unexpected characteristics of RSS values due to interference. To resolve this issue, we utilised the fingerprinting technique to ensure that unexpected characteristics of RSS values are taken into account when determining the user's position.

4.3.1 Training/Offline Phase

One of the important processes in the offline phase is the data acquisition. Firstly we will be developing an android application that will read the BLE signals from the designated BLE beacons and display the relevant information such as the MAC address and the RSS values. The main purpose of this application is to populate the fingerprinting database by inserting the reference point (Geographical Coordinate) and corresponding collected RSS values.

4.3.2 Positioning/Online Phase

During the online phase, the user's location will be determined through the collected RSS values from the user's device. The RSS values will then be extracted to form a vector which will be compared against the fingerprinted RSS values from the fingerprint database to determine the most viable reference point that will match the user's position. In a practical scenario, the collected RSS values from the user's device will not directly match with the trained RSS values in the fingerprint database therefore we will be using the K-Nearest Neighbours Algorithm (KNN).

For each reference point in the fingerprint database, we have the corresponding RSS vector, S where S is equivalent to $\langle S_1, S_2, S_3, \dots, S_n \rangle$. The RSS vector collected from the user's device will be U where U is equivalent to $\langle U_1, U_2, U_3, \dots, U_n \rangle$. For each vector S , we will be computing the euclidean distance between the vector U , this euclidean distance will be the numerical representation of the similarity between the RSS from the user's device and the RSS from a specific reference point in the fingerprint database. Figure , as shown below, describes the formula used to compute the Euclidean distance between the RSS vector of each reference point and the user's RSS vector.

$$d_i = \sqrt{\sum_{j=1}^{n_i} |SS_{rec,i_u}^j - SS_{DB,i}^j|}, \quad i \in I_{RP}$$

Figure 4.3.2.1: Euclidean Distance Formula (El Ashry & Sheta, 2019)

With the Euclidean distance for each reference point, the reference point will then be sorted in ascending order based on the Euclidean distance. A dynamic K metric will be determined based on the number of reference points with the smallest closely equivalent euclidean distance. The average of the K nearest neighbours will then be calculated with the weights assigned to each reference point, this weight will provide the level of influence over the result of the average (El Ashry & Sheta, 2019). For reference points that have lower Euclidean distance, a higher weight will be given, to reduce the effects of noise data on the average which represents the estimated user position.

4.4 Route Planning

4.4.1 Data Structure Parsing

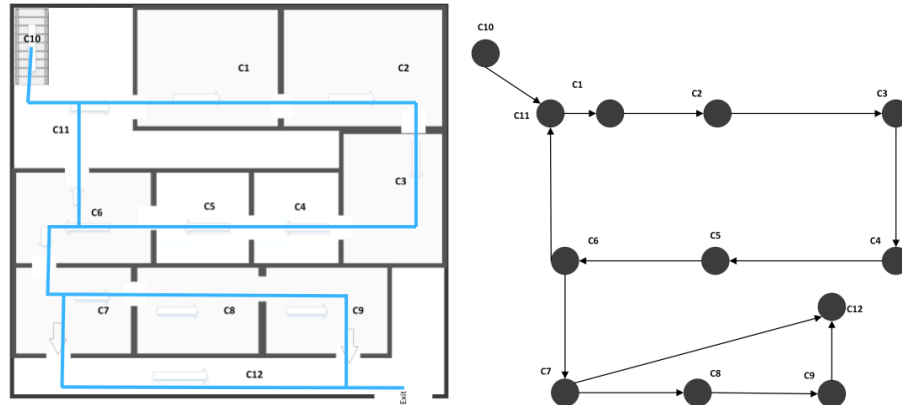


Figure 4.4.1.1: Conversion of Map Information to Graph Data Structure (Alamri, 2018)

With the route map created in the Map Planning component, we will be parsing the GeoJSON data to generate a graph data structure that simulates the real world environment. Each Point object will be parsed into a set of vertices and each adjacent Point ID referenced by the property of the current Point ID will represent an edge in the graph between the current Point object and the adjacent Point object.

4.4.2 Shortest Path Algorithm

Algorithm 1 Indoor Routing Algorithm

```

1: Insert a new element on the map with user's coordinate
2: Parse map into a graph data structure G
3: //Get source node and destination node from the parsed data structure
4: for each vertex V in G do
5:   if V is user's position node then
6:     //Source Node
7:     Store reference to V as S
8:   end if
9:   if V is user's destination node then
10:    //Destination Node
11:    Store reference to V as D
12:   end if
13: end for
14: Run Dijkstra's algorithm on graph G with source node S and destination
    node D
15: //Backtrack to obtain route
16: Let current be D
17: while current is not NULL do
18:   Track route
19:   Let current be the predecessor node of itself
20: end while
21: Return route

```

Figure 4.4.2.1: Pseudocode for Indoor Routing Algorithm

Before we can determine the shortest route, we have to identify the destination vertex in the graph data structure that will represent the user's destination of choice. This process will

involve querying the POI database to obtain the unique Point ID that represents the point of interest on the map representation. With this ID, we will traverse through the vertices of the graph data structure to determine the vertex that represents the destination node.

Another parameter that we have to identify before determining the shortest route is the source vertex of the graph data structure. The source vertex will represent the user's position and we know that the user's position is dynamic throughout the map, to resolve this problem an extra Point object with the user's coordinate is included in the map representation before parsing. Similar to the process of determining the destination vertex, we will traverse through the set of vertices in the graph data structure to obtain the source vertex.

We will then run Dijkstra's algorithm on the graph data structure from the source vertex to the destination vertex and perform backtracking to reconstruct the shortest route.

$$a = \sin^2(\phi B - \phi A/2) + \cos \phi A * \cos \phi B * \sin^2(\lambda B - \lambda A/2)$$

$$c = 2 * \text{atan2}(\sqrt{a}, \sqrt{(1-a)})$$

$$d = R * c$$

Figure 4.4.2.2: Haversine Formula

Dijkstra's algorithm works on graphs with weighted edges. We can represent the edge weights between each vertex of the parsed route map as the geographical distance between each point of interest. This way, we can accurately determine the shortest path from the source vertex to the destination vertex. To compute the geographical distance, we will be utilising the haversine formula to compute the great circle distance between 2 sets of geographical coordinates.

4.4.3 Deviation

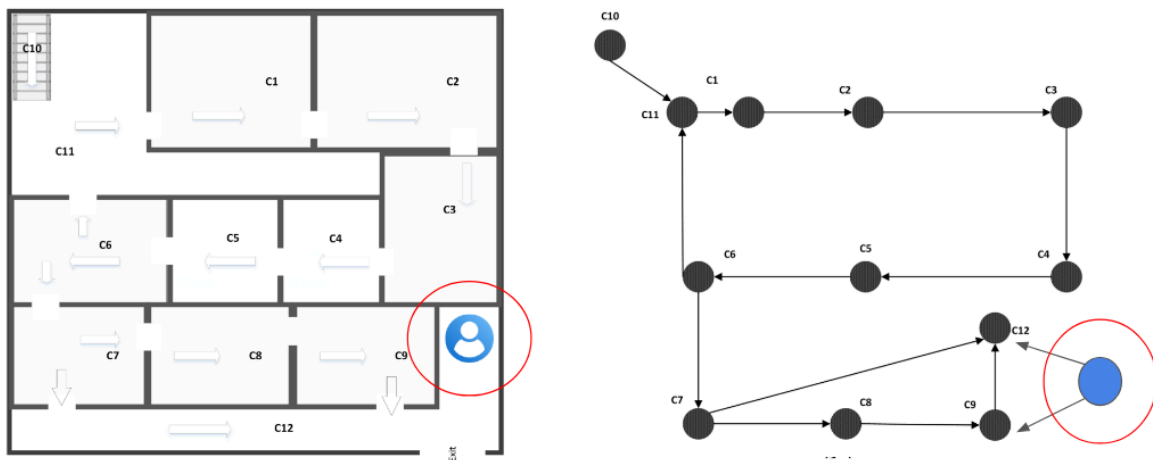


Figure 4.4.3.1: Incorporating Source Vertex (Alamri, 2018)

In our project proposal, we suggested the use of Breadth First Search algorithm because we wanted to populate the route map graph with all possible positions of the user. We realised that this is a very inefficient approach because the graph representation will be very convoluted. To solve this problem, we introduced dynamic source vertex integration; this methodology avoids over-populating the graph with vertices that inaccurately represent the distance. Instead, we can accurately represent the distance between each vertex as the edge weight which enables the use of Dijkstra's Algorithm to compute the shortest path.

4.5 Path Guiding

With the route data computed from the Route Planning component, we will be transforming that information into 2 different formats offered by our application. The first format is 2-Dimensional based where a navigation path will be displayed on a 2-Dimensional map for simplicity. The second format is Audio based where navigation instructions will be output from the user's device as an audio instruction to aid visually impaired users.

4.5.1 2-Dimensional Based



Figure 4.5.1.1: GeoJSON Line Object

In this approach, the collected route information from the Route Planning component will be used to create a path using the coordinates associated with each vertex of the route. This path will be represented as a Line object in the GeoJSON format and the collective coordinates together will form a path/line to the user's destination of choice.

4.5.2 Audio Based

$$\alpha_2 = \arctan2(\sin \alpha, -\sin U_1 \sin \sigma + \cos U_1 \cos \sigma \cos \alpha_1)$$

Figure 4.5.2.1: Vincenty's Formula (Vincenty, 1975)

To determine the sequence of directions the user must take from their position to the destination of choice. We firstly need to identify the forward azimuth to move from the user's current position to the next coordinate in the route. To compute the forward azimuth, we will be utilising the Vincenty's Formula with the user's position as a geographical coordinate and the next geographical coordinate of the computed route from the Route Planning component. This will provide us with the general cardinal direction that user's are expected to take, to move towards the next coordinate of the route.

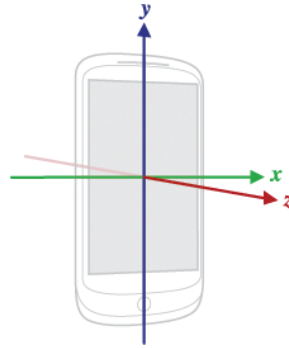


Figure 4.5.2.2: Android Geomagnetic Sensor

However, computing the forward azimuth is not sufficient in accurately determining the direction because users are not limited to a specific direction that they can face. To solve this problem, we need to compute the orientation of the user by leveraging on the geomagnetic sensors from the user's device to determine the cardinal direction that the user is facing.

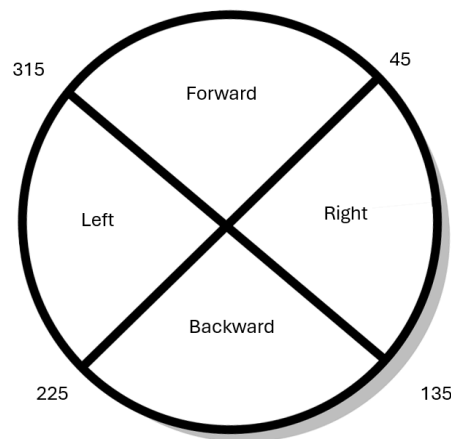


Figure 4.5.2.3: Mapping Azimuth Difference to Direction

With the cardinal direction of the user's orientation and forward azimuth. We can determine the direction by computing the difference between the user's orientation and forward azimuth.

4.5.3 Deviation

In our project proposal, we suggested 3 different forms of path guiding. Unfortunately, due to limited hardware infrastructure as mentioned in [section 3](#), we cannot complete the 3-Dimensional based navigation. Hence, it is not included in this final project report.

4.6 Software & Hardware Specification

This table provides a comprehensive overview of the technology stack that was used in the project. The left column outlines each component of our project and the right column states the technology used for that particular component.

Platform	Android
Frontend Language	Java
Backend Language	Python
Backend Framework	Flask
Mapping SDK	MapBox, MazeMap
Database	SQLite
IDE	Android Studio, VSCode
Testing Platform	Android Device with Geomagnetic Sensors and Bluetooth Support
Positioning Hardware	Bluetooth Low-Energy Beacons
Android Version	Android 11 and above
Version Control	Git
Code Repository	GitLab

Table 4.6.1: Software and Hardware Specifications of Final Project Implementation

5. Software Deliverables

This section outlines the core features of our application that will be delivered to end users, and briefly covers the device requirements needed to run the application. It also examines the software's quality by providing an overview of the strengths and weaknesses of various key analysis aspects, including robustness, security, usability, scalability, portability, and maintainability. This comprehensive evaluation highlights how these factors contribute to the overall effectiveness and reliability of the application for end users.

5.1 Summary of software deliverables

This subsection provides an overview of the functionalities of the application that users will experience. It includes detailed descriptions of key components, such as the user profile, navigation interface, and language options.

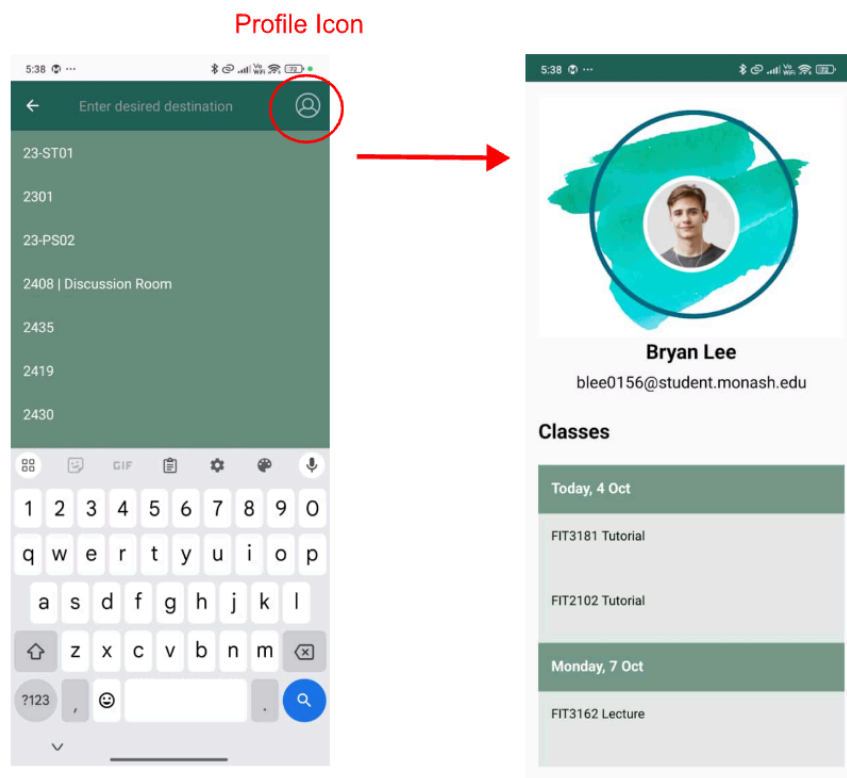


Figure 5.1.1: User's Profile

Our end product is an Android application designed to help users navigate seamlessly between rooms on floors 3 and 4 of Building 6. Users can log in with their student account to access [personalised class information](#) and room locations, or choose to log in as guests. Students with linked accounts can view their timetable and navigate directly to classrooms with a single click.

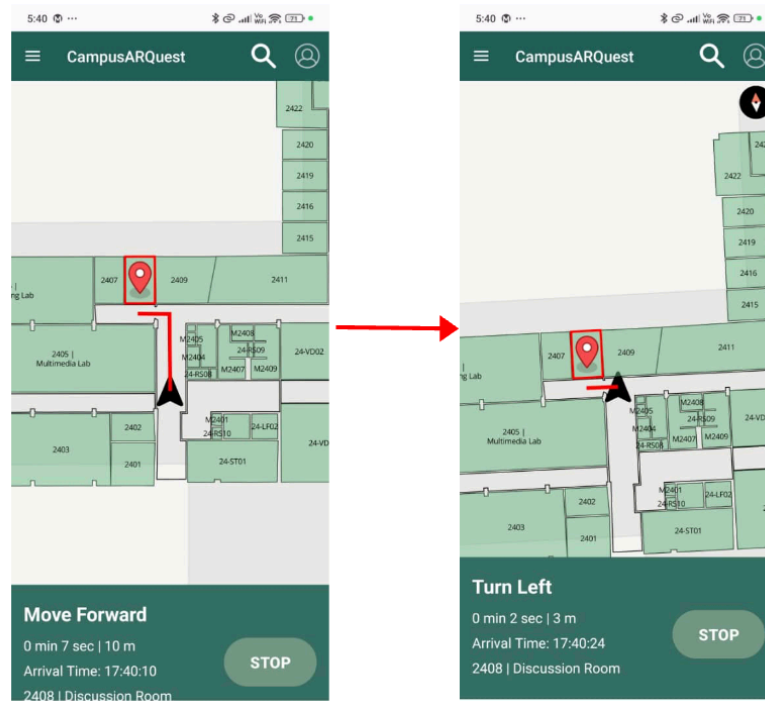


Figure 5.1.2: User's Navigation UI

The app provides real-time localization and route guidance in a 2D map view, where users are represented by an arrow indicating their [current position](#). When a destination is selected, a [red line](#) marks the shortest path from the user's current position to the destination. The route guidance is enhanced by specific prompts for orientation and movement, ensuring users can navigate easily even in complex indoor environments.

The UI displays remaining distance and [estimated arrival time](#), and users can terminate navigation at any point by tapping the "STOP" button on the progress bar.

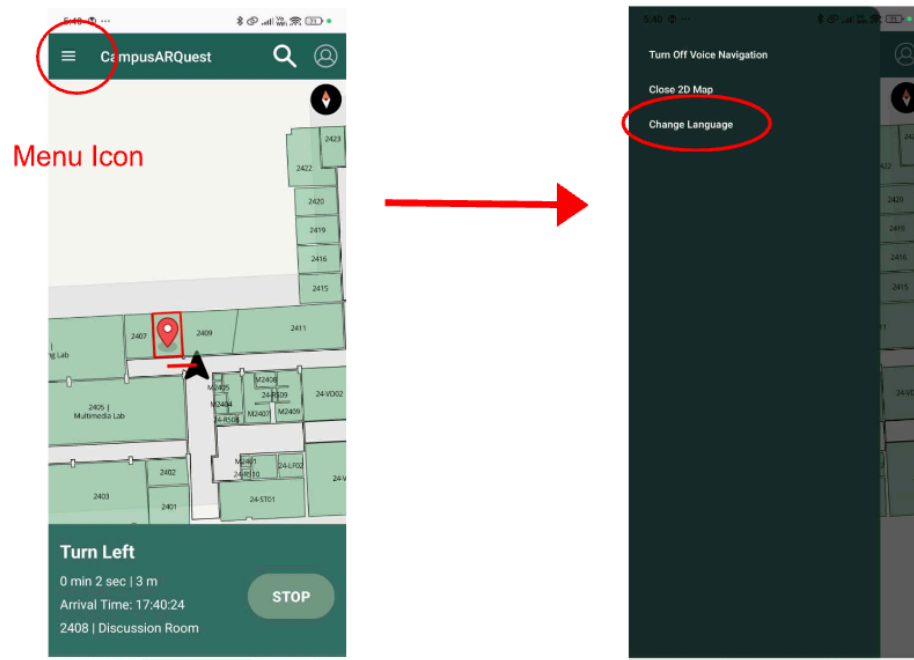


Figure 3.4.3: Switching Languages

[Voice navigation](#) is enabled by default, providing hands-free guidance that users can disable or switch to alternative languages.

Since most of the features were covered in [Section 3](#), we will not delve deeper in this section to avoid repetition.

However, we do have requirements on the user's device in order to fully enjoy the deliverables of our software. Table below outlines the system and hardware requirements, as well as the scope of the application.

Category	Specification
Software	
Operating System	Android
Android Version	11 or higher
Internet Connection	stable Wi-Fi connection
Hardware	
Gyroscope Sensor	Required
RAM	Minimum of 4GB
Scope	
Application Usage	Designed to function exclusively within Monash Campus
Location	Building 2, Levels 3 and 4

Table 5.1.1: Basic Requirements on User's End

In conclusion, this section has outlined the key deliverables of our software, highlighted essential requirements on the user's device, and presented an overview of the application's quality across various key dimensions. These details ensure a clear understanding of how the end product will function and what users can expect in terms of usability and experience.

5.2 Software Quality Summary

This section evaluates the software's quality by focusing on a few key attributes. Each attribute is crucial to ensuring a smooth user experience, reliable functionality, and adaptability to evolving requirements. By assessing the good and down sides of each quality aspect, this section provides a comprehensive overview of the application's strengths and limitations.

5.2.1 Robustness

Robustness refers to the software's resilience against various errors, unexpected inputs, and operating conditions. It's a critical aspect of software quality, especially for applications handling dynamic data in real-time, like our indoor navigation app.

Good Sides:

- **Error Handling:**

The app has extensive error handling for common issues, such as signal loss from BLE beacons. If a beacon signal is weak or lost, the application relies on fallback calculations using neighbouring beacons, ensuring continuous tracking. Hence, when a beacon is faulty, the app will not crash or suddenly detect the user in another hallway. The prediction might be slightly differ, but will not cause a big issue overall.

- **Crash Prevention:**

The application incorporates multiple validation checks to prevent crashes caused by unexpected inputs or signal fluctuations. For example, the input location data is checked, and will only proceed if it is in one of the valid locations covered by the map. On top of that, if a user walks out of the covered floors or area, the app safely defaults to the closest possible point, avoiding abrupt termination.

Shortcomings:

- **Signal Variability:**

The application is not robust to different environmental factors. BLE signals are sensitive to interference from objects or walls. If there are obstructions, the localization accuracy can be compromised, affecting robustness in densely constructed areas.

- **Performance Under Load:**

During high traffic, the app tends to fail in maintaining steady performance, which is especially compounded by crowded environments, as it could not handle fluctuations in multiple user signals. In cases like this, the app's real-time calculations for location updates, rerouting, and orientation adjustments may occasionally impact system resources on lower-end devices, causing delays or slight lags.

5.2.2 Security

Security ensures data protection and the prevention of unauthorised access, which is critical when handling personal information and sensitive data.

Good Sides:

- **Authentication:**

Users can choose to use the application as a guest or log in as a student. For protecting students' personal information, the app integrates a secure login process for student accounts, utilising the 2-factor authentication to ensure that only authorised users can access specific features such as class schedules.

- **Data Encryption:**

Sensitive data, such as user locations, account details, and query history, is encrypted in the app's local SQLite storage. This ensures that even if a device is compromised, unauthorised access to raw data is prevented. For instance, using SQLCipher or other encryption layers, SQLite can store data in an encrypted format, providing security for data stored offline on the device.

When transmitting sensitive data to backend servers or cloud services, all data is encrypted using TLS, a standard protocol for secure internet communication. This prevents interception of user location and account details by potential attackers while data travels across networks.

When a user's location or query history is sent from the app to the server for route calculation, the TLS encryption prevents third parties from intercepting and viewing this information, ensuring the privacy and integrity of user data.

- **Access Control:**

Only team members with specific roles or permissions can access backend services that handle sensitive user data. Role-based access limits data exposure to only those who require it for their function.

User data is anonymized in logs to protect identity, ensuring that access to personal information is limited to essential functions only. Logs can retain data useful for analysis (such as timestamps or location coordinates) without linking it to user identities.

For development or analytics, a developer may access anonymized location data to improve route predictions without viewing individual user identities or precise locations, ensuring privacy.

Shortcomings:

- **Permissions Management:**

While the app collects location data to enhance navigation features, users may feel the need for better control over what data is stored and for how long. Allowing users to delete past location data, manage what data is logged, and receive regular updates about how their data is protected would address some of these concerns.

To mitigate this issue, providing a “Delete My Data” option or customizable data retention settings could give users more confidence and control, reducing the risk of unease about continuous tracking.

- **Data Security Risks:**

While access is restricted to authorised personnel, the backend’s security protocols should be reviewed regularly to minimise potential breaches. A single compromised account could expose sensitive data, so frequent access audits and multifactor authentication are essential.

If a team member’s access credentials are compromised, an attacker could potentially access anonymized data logs, risking data misuse even in an anonymized form.

5.2.3 Usability

Usability is a key factor for an app designed for quick and efficient navigation, and it evaluates how easily users can interact with the software. Good usability ensures that users of varying technical levels and abilities can engage with the app without difficulty. By providing a seamless, accessible, and efficient user experience, the app promotes user satisfaction and meets essential navigation needs.

Good Sides

- **Intuitive Design:**

The app boasts a clean, minimalistic interface, with clearly labelled options and straightforward icons, allowing new users to navigate without extensive guidance. Users can quickly identify their options and make selections with minimal input, which enhances the overall user experience.

Buttons for “Start Navigation” and “End Navigation” are easily recognizable and placed prominently, allowing users to control navigation easily, which is essential in stressful or crowded environments where quick decision-making is necessary.

- **Language Support:**

Multi-lingual support in the app broadens accessibility for users who prefer or require languages other than English. This makes the app adaptable to diverse environments and user groups, improving its inclusivity.

A non-native English speaker can select their preferred language, making it easier to follow instructions without struggling with language barriers.

- **Real-Time Feedback:**

Real-time voice guidance and automatic rerouting upon deviations allow users to adjust quickly without studying the map or trying to pinpoint their exact location on a small screen. This enhances navigation efficiency and reduces the chances of users becoming lost.

If a user accidentally moves off the suggested path, the app recalculates the route and provides immediate audio cues, allowing them to get back on track without pausing to study the map.

Shortcomings

- **Accessibility for Differently Abled Users:**

The app could enhance its accessibility for visually impaired users. Although it includes voice guidance, additional text-to-speech features, or compatibility with screen readers, it would further support visually impaired individuals by vocalizing interface options and ensuring compliance with WCAG standards.

Visually impaired users might benefit from detailed verbal descriptions of nearby points of interest, dynamic directional guidance, or text-to-speech support for menu options. Without these, they may struggle to navigate or interact with the app as effectively as sighted users.

5.2.4 Scalability

Scalability in this context is the system's ability to support growth in user volume, geographic area, or feature set without a decline in performance. An ideal scalable system handles increased demand without excessive adaptation effort, enabling the app to expand effectively in user reach and geographic coverage.

Good Sides

- **Dynamic Routing through Graph-based Paths:**

Unlike static routing systems that depend on predefined paths, this application leverages a graph structure combined with Dijkstra's algorithm for real-time route calculations. This structure means that expanding the navigable area or map requires only the addition of new vertices (representing points) and edges (representing paths) to the graph. Dijkstra's algorithm then dynamically calculates the shortest path for any given user request without manual intervention, making the routing system inherently flexible and easily adaptable to growth.

If a new building wing is added to the facility, it can be incorporated into the navigation system by updating the graph with new nodes and connections rather than redesigning the entire routing logic. This approach saves time and resources as the map grows.

- **Automated Path Updates:**

The routing mechanism does not rely on hardcoded paths, which minimizes manual recalibration. This is particularly advantageous when scaling across larger spaces or adding new points of interest, as the system will automatically identify the shortest paths on the updated map.

In a university campus setting, new lecture halls or facilities can be added by simply defining their coordinates in the graph, allowing the system to automatically integrate these locations without extensive programming or route reconfiguration.

Shortcomings

- **BLE Beacon Infrastructure Requirements:**

The app's reliance on Bluetooth Low Energy (BLE) beacons for user localization presents scalability limitations in physical setup. Each expansion to a new building or area requires deploying additional BLE beacons, which may involve significant investment in both hardware and installation.

Expanding the navigation system to a new floor or building requires careful beacon placement to ensure accurate localization, which could involve logistical challenges in large or complex facilities.

- **Dependence on Fingerprinting Localization:**

As noted in [section 3.6](#), the current fingerprinting-based localization approach may become inefficient in larger spaces or in areas with high user density. For broader coverage, switching to a more scalable localization technique, such as triangulation, may be necessary to maintain quick response times and reduce setup complexity.

In a large convention centre with multiple sections, fingerprinting every few metres becomes labour-intensive and may result in slower location determination, making triangulation a more scalable alternative for accurate, quick localization.

- **Server Load with Increased User Demand:**

The app's real-time location processing and dynamic rerouting impose a considerable load on backend servers, particularly as user numbers increase. While the current system may handle moderate traffic, significant growth could lead to delays or require additional server resources, impacting the system's overall scalability.

During peak times, such as the beginning of a university semester when many students are navigating campus, the servers may experience a surge in processing requests, potentially causing bottlenecks that could slow down response times without upgraded server infrastructure.

5.2.5 Portability

Portability in software refers to the ability of an application to function across various environments and platforms with minimal modifications. This attribute is essential for maximising user accessibility and reaching a broader audience.

Good Sides

- **Cloud-Based Architecture:**

The application leverages cloud services, reducing dependencies on specific platforms or local infrastructure. This cloud-centric approach allows for easy updates, maintenance, and deployment, as the software can be hosted and accessed from different locations without significant reconfiguration.

If the app is updated with new features or improvements, these changes can be deployed on the server side, making them instantly available to all users without requiring them to download updates manually. This streamlines the user experience and ensures consistency across all devices.

Shortcomings

- **Cross-Platform Support:**

Currently, the application is limited to Android devices, which restricts its usability for users on other platforms, such as iOS or web-based interfaces. This limitation can alienate potential users who prefer different operating systems.

A user with an iPhone may be unable to access the navigation features, forcing them to rely on alternative solutions or competing apps that support multiple platforms.

- **Dependency on Device Hardware:**

Certain functionalities, such as orientation sensing, rely on specific hardware components like gyroscopes and accelerometers. This dependence means that performance can vary significantly across different devices. For instance, newer smartphones typically have more advanced sensors, while older or budget models may lack precision or may not support these features at all.

A user with an older Android device may experience inconsistent orientation tracking, leading to inaccurate navigation guidance, which could frustrate users and affect the app's reliability.

- **Limited Compatibility with Low-End Devices:**

The app's real-time processing and Bluetooth requirements may not perform optimally on low-end or older devices. This can limit accessibility for users who cannot upgrade their hardware, potentially leaving out a segment of the audience who could benefit from the navigation system.

In a university setting, students using older smartphones might struggle with the app's functionality, such as receiving real-time updates or processing location data quickly, causing delays in navigation and an overall diminished experience.

5.2.6 Maintainability

Maintainability refers to how easily software can be modified to fix defects, improve performance, or adapt to changing requirements. This characteristic is vital for ensuring that an application remains functional and relevant over time, allowing developers to implement updates without introducing new issues.

Good Sides

- **Well-Organized Codebase:**

The application utilises modular and component-based coding practices, which facilitate easy navigation and understanding of the code structure. By encapsulating specific features within dedicated classes—such as the BLE detection class, point of interest class, and enumerations for directions—developers can quickly locate and address the relevant code segments.

If a bug arises in the BLE detection functionality, developers can directly access the BLE detection class without sifting through unrelated code, streamlining the debugging process and enabling quicker fixes.

```
public class PointOfInterest {
    private int id;
    private String identifier;
    private double latitude;
    private double longitude;
    private String name;
    private int z;
```

Table 5.2.6.1: Modularised Point of Interest Class

- **Documentation:**

Comprehensive documentation accompanies the codebase, providing insights into functionality, code structure, and usage instructions. This resource aids developers in understanding the application's architecture, thus reducing the time needed for onboarding new team members or for existing developers to implement changes.

When a new feature is requested, developers can refer to the documentation to understand how to integrate it seamlessly with existing functionality, ensuring that the feature is compatible with the current architecture.

- **Continuous Integration (CI):**

The inclusion of automated testing as part of the continuous integration process enhances maintainability by allowing developers to detect and address issues swiftly during the development cycle. This proactive approach minimises the risk of bugs being introduced into the production environment.

With each update to the codebase, the CI pipeline runs tests that automatically verify whether existing features function as expected, alerting developers to any failures immediately. This process allows for quick iterations and reduces the time spent on debugging in the long run.

Shortcomings

- **Dependency on BLE Technology:**

As Bluetooth Low Energy (BLE) standards evolve, the application may face challenges in maintaining compatibility with newer standards or devices. This reliance on external technology means that significant updates may be required to ensure that the beacon-based system continues to function correctly.

If a new BLE standard is introduced that changes how devices communicate, developers might need to overhaul the existing BLE detection class, potentially leading to extensive modifications and increased maintenance effort.

- **Complexity of Real-Time Features:**

The real-time aspects of localization and rerouting introduce additional complexity to the codebase. This complexity can make it more challenging to maintain and update, as changes to one part of the system might unintentionally affect others, necessitating extensive testing to ensure system stability.

If a developer needs to modify the algorithm used for real-time rerouting, they must thoroughly test not only the rerouting functionality but also any related components that might interact with the localization system, increasing the risk of bugs and the overall time required for updates.

In conclusion, the analysis of software quality demonstrates a balanced understanding of the application's strengths and areas needing improvement. While the application excels in areas such as usability, security, and real-time routing, it also faces challenges related to signal variability, scalability with BLE infrastructure, and accessibility for differently abled users. Addressing these shortcomings through proactive improvements and thoughtful updates will further enhance the application's effectiveness and reliability, supporting its long-term success and user satisfaction.

5.3 Sample Source Code

This section demonstrated our code for detecting Bluetooth signals to meet the requirement of updating the user's position accurately. The relevant code has been included in the [Appendix](#) for reference.

6. Software And Project Critique

This section describes both the success and failures of our project execution and provides reasoning of changes from our realised project outcome against the initial project proposal.

6.1 Discussion of Project Execution

6.1.1 Success

Overall, the execution of the project is successful because we were able to produce a deliverable that meets the majority of the expected requirements that were stated at the start of the project. On top of that, our team established a strong relationship with the stakeholder by ensuring that project updates were frequently communicated with the stakeholder which enabled our team to continuously refine the project requirements throughout the duration of the project and satisfy the updated expectations of the stakeholders by producing a deliverable that exceeds the initial requirements stated for the project.

One of the main reasons that contributed towards the success of our project was the thorough scheduling activity that our team held at the beginning of the project. With careful consideration of scheduling, we were able to accurately break down and schedule works/tasks to avoid dependency problems. For instance, the User Positioning and Route Planning component can only begin once the Map Planning component has been completed. This allowed our team to efficiently complete a lot of the scheduled works/tasks ahead of the expected schedule. In addition, we designed the scheduling of our project to be more flexible in accommodating tasks that pose a higher risk in facing unforeseen circumstances. For instance, we were facing a persistent bug on our User Positioning component. Thankfully, we identified the tasks that fall under the User Positioning component as the task with “higher risk in facing unforeseen circumstances”. Therefore, our schedule provided us with more time to fix those bugs while maintaining the expected progress timeline.

Setting realistic goals are key factors that determine the success of projects. During the planning phase of the project, we ensured that the outlined scope aligns with the objectives of our project. This enabled the team to focus on completing features that provided the most benefit to the stakeholders and prevented the team from side-tracking on “nice to have” features that may potentially delay the progress timeline. In addition, our scope is carefully designed based on the available resources of the team. For instance, many of the team members have external commitments outside the project and setting the scope of the project to accommodate the entire campus was not feasible given the resources that were available to us. This allowed the team to successfully produce more refined and important deliverables that can benefit stakeholders.

6.1.2 Failures

Despite the overall success of the project, we faced 2 significant shortcomings from our final project outcome: the user positioning feature did not meet the expected performance as stated in the requirements and our application lacked a 3-Dimensional based navigation guide. The issues arose due to insufficient consideration of limited hardware infrastructure in

our risk management plan. Specifically, we did not account for the possibility of slow broadcast frequency from the BLE beacons and the level of augmented reality SDK support on the Android device model that we tested with.

While our project schedule was well executed, we could have optimised task efficiency and allocated more time for developing additional features that can enhance our final deliverable. This improvement could have been achieved by refining our task allocation strategy, as we experienced significant downtime during development, with team members waiting for specific tasks to be completed before they can begin with others. Breaking down complex tasks into smaller subproblems and evenly distributing them among team members instead of allocating a complex task for each team member would have enabled faster completion and reduced the delays between dependent tasks.

In terms of our project management methodology, we struggled to consistently adhere to essential Agile activities such as the sprint reviews and retrospective. This inconsistency limited our ability to make iterative improvement after each sprint. Consequently, issues such as the accuracy of the user positioning model weren't adequately addressed after the end of each sprint, leading to unmet requirements and negatively impacting the overall quality of our project objectives.

6.2 Comparison of Realised Project to Initial Project Proposal

As mentioned from the methodology section and the failures of our project execution, we initially proposed the use of WiFi access points as the choice of our indoor positioning technology. However, due to limitations over the positioning of the access points. Our team decided to utilise the bluetooth beacons approach to ensure that our beacons can be placed in designated areas that can improve the overall accuracy of our user positioning component.

However, utilising the bluetooth beacons came with its limitations. In our risk management plan, we did not adequately account for potential issues relating to limited hardware infrastructure. Specifically, the beacons that we have obtained for our project did not meet the expected broadcast frequency requirements of 10ms~50ms; instead, the base broadcast frequency of the beacons operated at 500ms. This discrepancy prevented us from achieving the expected initial requirement of updating the users position within an interval of 5s. This issue could have been resolved if we carefully outlined a resolution plan of encountering limited hardware infrastructure scenarios so that we would have enough time to obtain hardware replacements that would meet our project requirements.

In the project management plan of our initial project proposal, we carefully outlined Agile activities for the team to undertake to ensure that our project goals were met. However, during the execution of our project, the team diverged from the plans due to external commitments, focusing only on essential activities that our team deemed to be essential. This deviation resulted in certain requirements not meeting the expected standards of our project proposal. To address these issues, we should have maintained discipline and recognized the importance and value of consistently following all Agile activities rather than selectively modifying the methodology based on what we feel is important.

7. Conclusion

All in all, this report presented the development of an indoor navigation system that leverages Bluetooth Low Energy (BLE) beacons for real-time positioning, allowing for accurate tracking of users' movements on a dynamic 2D map. In the [Project Background section](#), we provided an overview of this system, highlighting how it overcomes the limitations of static maps. By utilising smartphone geomagnetic sensors, the system delivers personalised navigation instructions based on user orientation. Additionally, Dijkstra's algorithm was implemented for efficient shortest route calculation.

In the [Outcome section](#), we discussed the features implemented and how they related to our initial project requirements. Next, we demonstrated how each feature worked with screenshots of our application. We then linked these features back to our Requirements Traceability Matrix (RTM) to show which requirements were met and how they were fulfilled. For any requirements that weren't met or were modified, we provided justifications. After that, we examined the app's limitations and shortcomings as a whole. Then, we explored potential future work to build on what we achieved.

In the [Methodology section](#), we carefully outlined the 4 key components of our system architecture: Map Planning, User Positioning, Route Planning and Path Guiding. We delved into the details on what was performed at each of the key components such as the algorithms, formulas and methodologies. For each key component, we articulated the deviations that were improvised from the initial project proposal and finally we stated the software and hardware tools that were required to execute the methodology.

In the [Software Deliverables section](#), we gave a clear rundown of the main features our app offers and what users need in terms of device requirements to get the best experience. We also dove into key aspects like robustness, security, usability, scalability, portability, and maintainability, highlighting both the positives and potential challenges. By looking at these factors, we can see how they play a role in making the app reliable, user-friendly, and flexible enough to adapt to different situations and user needs.

In the [Software and Project Critique section](#), we demonstrated the successes and failures of the project along with the reason behind them by referencing the project management concepts and methodologies. Additionally, we compared the deviations of our finalised project outcome against the initial project proposal to illustrate the reasons behind those deviations and what could have been improved in hindsight to avoid the failures that we have experienced during the project execution.

To conclude, our project has largely met our expectations by successfully fulfilling and even exceeding most of the initial requirements. However, we regret that the AR feature for camera view could not be implemented within this phase. While the other features have been tested and proven to function as intended, there are still potential issues that need to be addressed if the project is to be refined and expanded in future stages. Moving forward, future work could concentrate on resolving the identified shortcomings and enhancing the existing system, as well as incorporating additional functionalities like customizable features mentioned in [Section 3.6](#). This would help in elevating the project to a more advanced and polished version, providing an even better user experience.

8. References

- Alamri, S. (2018). An efficient shortest path routing algorithm for directed indoor environments. *ISPRS International Journal of Geo-Information*, 7(4), 133. <https://doi.org/10.3390/ijgi7040133>
- Brena, R. F., García-Vázquez, J. P., Galván-Tejada, C. E., Muñoz-Rodríguez, D., Vargas-Rosales, C., & Fangmeyer, J. (2017). Evolution of indoor positioning technologies: A survey. *Journal of Sensors*, 2017, 1–21. <https://doi.org/10.1155/2017/2630413>
- Distance on a sphere: The Haversine formula*. Esri Community. (2021b, December 12). <https://community.esri.com/t5/coordinate-reference-systems-blog/distance-on-a-sphere-the-haversine-formula/ba-p/902128>
- El Ashry, A. E., & Sheta, B. I. (2019). Wi-Fi based indoor localization using trilateration and fingerprinting methods. *IOP Conference Series: Materials Science and Engineering*, 610(1), 012072. <https://doi.org/10.1088/1757-899x/610/1/012072>
- Faragher, R. M., & Harle, R. K. (2015). Towards an Efficient, Intelligent, Opportunistic Smartphone Indoor Positioning System. *Navigation*, 62(1), 55–72. Portico. <https://doi.org/10.1002/navi.76>
- Hart, P., Nilsson, N., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107. <https://doi.org/10.1109/tssc.1968.300136>
- Keerthana, R. B., Priyadarshini, G., Vasudevan, S. K., Shree, H., & Venkatachalam, K. (2020). An intelligent and interactive AR-based location identifier for indoor navigation. *International Journal of Advanced Intelligence Paradigms*, 15(1), 32. <https://doi.org/10.1504/ijaip.2020.10025743>
- Li, Y., Zhuang, Y., Lan, H., Zhou, Q., Niu, X., & El-Sheimy, N. (2016). A Hybrid WiFi/Magnetic Matching/PDR Approach for Indoor Navigation With Smartphone Sensors. *IEEE Communications Letters*, 20(1), 169–172. <https://doi.org/10.1109/lcomm.2015.2496940>
- Lie, K. K., Yeo, K. S., Ngoh Ting, A. K., & Tze Chieng, D. H. (2020). Indoor Tracking with Bluetooth Low Energy Devices Using K-Nearest Neighbour Algorithm. 2020 IEEE 10th Symposium on Computer Applications & Industrial Electronics (ISCAIE). <https://doi.org/10.1109/iscaie47305.2020.9108790>
- Makariye, N. (2017). Towards shortest path computation using Dijkstra algorithm. 2017 International Conference on IoT and Application (ICIOT). <https://doi.org/10.1109/iciota.2017.8073641>
- Pasricha, S. (2020). Overview of Indoor Navigation Techniques. *Position, Navigation, and Timing Technologies in the 21st Century*, 1141–1170. Portico. <https://doi.org/10.1002/9781119458555.ch37>

- Sensor types* : *Android Open Source Project*. Android Open Source Project. (n.d.).
<https://source.android.com/docs/core/interaction/sensors/sensor-types>
- Ta, V.-C., Dao, T.-K., Vaufreydaz, D., & Castelli, E. (2018). Smartphone-Based User Positioning in a Multiple-User Context with Wi-Fi and Bluetooth. 2018 International Conference on Indoor Positioning and Indoor Navigation (IPIN).
<https://doi.org/10.1109/ipin.2018.8533809>
- Vincenty, T. (1975). Direct and inverse solutions of geodesics on the ellipsoid with application of nested equations. *Survey Review*, 23(176), 88–93.
<https://doi.org/10.1179/sre.1975.23.176.88>
- Zhang, P., Zhao, Q., Li, Y., Niu, X., Zhuang, Y., & Liu, J. (2015). Collaborative WIFI fingerprinting using sensor-based navigation on smartphones. *Sensors*, 15(7), 17534–17557. <https://doi.org/10.3390/s150717534>

9. Appendix

```

54 public class MainActivity extends AppCompatActivity { ±jcho0156+4*
55
535 protected void onResume() { ±jcho0156+1*
536     // Log a message indicating that the activity has resumed
537     Log.d(tag: "state", msg: "resume");
538
539     // Call the superclass's onResume method to ensure that the activity
540     // lifecycle is properly managed by the Android framework
541     super.onResume();
542
543     // Start the BLE scanning process to discover nearby Bluetooth devices
544     this.bleScanner.startScan();
545
546     // Check if the orientation sensor has been initialized and start it
547     // to begin receiving updates about the device's orientation
548     if (this.orientationSensor != null) {
549         this.orientationSensor.startSensor();
550     }
551 }

```

Figure 9.1: Call Function to Scan Beacon Signal

```

31 public class BLEScanner { 38 usages ±jwon0147+4*
32
302 public void startScan() { 1 usage ±jwon0147+1*
303     // Check if a scan is not already in progress
304     if (!this.scanning) {
305         // Set scanning flag to true to indicate that a scan is in progress
306         this.scanning = true;
307
308         // Check if the BLUETOOTH_SCAN permission is granted
309         if (ActivityCompat.checkSelfPermission(this.context, permission: "android.permission.BLUETOOTH_SCAN") != 0) {
310             // Request the BLUETOOTH_SCAN permission from the user
311             ActivityCompat.requestPermissions((Activity) this.context, new String[]{"android.permission.BLUETOOTH_SCAN"}, requestCode: 5);
312
313             // Check again if the permission is still not granted after the request
314             if (ActivityCompat.checkSelfPermission(this.context, permission: "android.permission.BLUETOOTH_SCAN") != 0) {
315                 // Log a debug message if permission is still not granted
316                 Log.d(tag: "what", msg: "what");
317             }
318         }
319
320         // Start the BLE scanning process using the BluetoothLeScanner
321         this.bluetoothAdapter.getBluetoothLeScanner().startScan(this.scanCallback);
322
323         // Log the action indicating that the BLE scan has started
324         Log.d(TAG, msg: "BLE scan started");
325     }
326 }

```

Figure 9.2: Scanning Bluetooth Signal

```

31 public class BLEScanner { 38 usages ± jwon0147+4*
46     private final ScanCallback scanCallback = new ScanCallback() { 2 usages ± jwon0147+2*
47         public void onScanResult(int callbackType, ScanResult result) { no usages ± jwon0147+2*
74
75         // Check if the current destination is null and routing is active, or if a destination exists but routing is not active
76         if ((BLEScanner.this.mainActivity.getCurrentDestination() == null && BLEScanner.this.isRouting.booleanValue()) || (BLEScanner.this
77             .mainActivity.getCurrentDestination() != null && !BLEScanner.this.isRouting.booleanValue())) {
78             // Call the updateRouteUI method to refresh the route UI
79             // Use the latitude, longitude, and floor level (z) from the previous prediction's MAC information
80             updateRouteUI(
81                 mainHandler,
82                 previousPrediction.getMac().getLatitude(), // Get latitude from the previous prediction's MAC
83                 previousPrediction.getMac().getLongitude(), // Get longitude from the previous prediction's MAC
84                 previousPrediction.getMac().getZ() // Get the floor level (z) from the previous prediction's MAC
85             );
86     }
}

```

Figure 9.3: Call Function to Display User's Route on the UI

After scanning, we will display user's position on the UI

```

31 public class BLEScanner { 38 usages ± jwon0147+4*
46     private final ScanCallback scanCallback = new ScanCallback() { 2 usages ± jwon0147+3*
89     public void updateRouteUI(Final Handler handler, Double latitude, Double longitude, final int z) { 2 usages ± jwon0147+2*
105
106     // Set routing status to true since we have a valid destination
107     Boolean unused2 = BLEScanner.this.isRouting = true;
108
109     // Call the navigation API to get the route using the current destination ID and coordinates
110     BLEScanner.this.navApiInterface.getRoute(
111         BLEScanner.this.mainActivity.getCurrentDestination().getId(),
112         longitude.doubleValue(),
113         latitude.doubleValue(),
114         z
115     ).enqueue(new Callback<ResponseBody>() { ± jwon0147+2*
116         // This method is called when the route request is successful
117         public void onResponse(Call<ResponseBody> call, Response<ResponseBody> response) { ± jwon0147+2*
118             try {
119                 // Parse the response body to get route data as JSON
120                 JSONObject routeData = (JSONObject) new Gson().fromJson(response.body().string(), JSONObject.class);
121                 // Extract the route array for the specified floor level (z)
122                 JSONArray route = routeData.get("route").getAsJsonObject().get(Integer.toString(z)).getAsJSONArray();
123                 // Get the total distance of the route
124                 int distance = routeData.get("distance").getAsInt();
125
126                 String directionString = ""; // Variable to hold direction instructions
127
128                 // Check if direction data is present in the response
129                 if (!routeData.get("direction").isJsonNull()) {
130                     // Get the X direction based on the provided orientation sensor data
131                     Direction xDirection = Direction.getDirectionX(routeData.get("direction").getAsDouble(), orientationSensor.getOrientation());
132                     // Log the latitude and longitude for debugging
133                     Log.d(TAG, "msg: Double.toString(latitude) + " | " + Double.toString(longitude));
134                     // Provide navigation instruction based on direction
135                     provideNavigationInstruction(xDirection);
136                     directionString = xDirection.toString(); // Convert direction to string
137                 }
138             }
139         }
140     });
141 }

```

Figure 9.4: Function to Calculate and Display the Shortest Route.

This function includes determining directional guidance for each navigation step, and activating voice navigation cues while displaying the shortest route possible.


```

31 public class BLEScanner { 38 usages ± jwon0147 +4*
46     private final ScanCallback scanCallback = new ScanCallback() { 2 usages ± jwon0147 +3*
89         public void updateRouteUI(final Handler handler, Double latitude, Double longitude, final int z) { 2 usages ± jwon0147 +2*
114             }.enqueue(new Callback<ResponseBody>() { ± jwon0147 +2*
116                 public void onResponse(Call<ResponseBody> call, Response<ResponseBody> response) { ± jwon0147 +2*
137
138                     // Check if floor data is present in the response
139                     if (!routeData.get("floor").isJsonNull()) {
140                         // Get the Y direction based on the floor number
141                         Direction yDirection = Direction.getDirectionY(routeData.get("floor").getAsInt());
142                         directionString = yDirection.toString(); // Update direction string
143                     }
144
145                     // Initialize a list to store the route coordinates for the floor
146                     List<List<Double>> floorRoute = new ArrayList<>();
147                     // Iterate through the route coordinates from the JSON array
148                     Iterator<JsonElement> it = route.iterator();
149                     while (it.hasNext()) {
150                         JsonArray coordinate = it.next().getAsJsonArray();
151                         // Add the coordinates (latitude, longitude) to the floor route list
152                         floorRoute.add(new ArrayList<>(List.of(
153                             Double.valueOf(coordinate.get(0).getAsDouble()),
154                             Double.valueOf(coordinate.get(1).getAsDouble())
155                         )));
156                     }
157
158                     // Create final variables to pass to the UI update
159                     final String finalDirectionString = directionString; // Direction instructions
160                     final String finalTimeAndDistance = "2 min" + " | " + distance + " m"; // Placeholder for time and distance
161                     final String finalArrivalTime = "Arrival Time: 12:34pm"; // Placeholder for arrival time
162
163                     // Post a runnable to update the UI on the main thread
164                     handler.post(() -> {
165                         // Update the bottom sheet info with direction, time, and arrival details
166                         MainActivity.updateBottomSheetInfo(finalDirectionString, finalTimeAndDistance, finalArrivalTime);
167                         // Remove the previous route from the 2D view fragment
168                         BLEScanner.this.twoDViewFragment.removeRoute();
169                         // Add the new route to the 2D view fragment
170                         BLEScanner.this.twoDViewFragment.addRoute(floorRoute);
171                     });

```

Figure 9.5: Updates the progress bar to show real-time navigation status.